What's New in Python

Release 3.13.7

A. M. Kuchling

October 01, 2025

Python Software Foundation
Email: docs@python.org

Contents

1 Summary – Release Highlights **3**

2 New Features **5**
2.1 A better interactive interpreter . . . 5
2.2 Improved error messages . . . 5
2.3 Free-threaded CPython . . . 6
2.4 An experimental just-in-time (JIT) compiler . . . 7
2.5 Defined mutation semantics for `locals()` . . . 8
2.6 Support for mobile platforms . . . 8

3 Other Language Changes **9**

4 New Modules **10**

5 Improved Modules **10**
5.1 argparse . . . 10
5.2 array . . . 10
5.3 ast . . . 10
5.4 asyncio . . . 11
5.5 base64 . . . 11
5.6 compileall . . . 11
5.7 concurrent.futures . . . 12
5.8 configparser . . . 12
5.9 copy . . . 12
5.10 ctypes . . . 12
5.11 dbm . . . 12
5.12 dis . . . 12
5.13 doctest . . . 13
5.14 email . . . 13
5.15 enum . . . 13
5.16 fractions . . . 13
5.17 glob . . . 13
5.18 importlib . . . 13
5.19 io . . . 14
5.20 ipaddress . . . 14
5.21 itertools . . . 14
5.22 marshal . . . 14
5.23 math . . . 14

5.24 mimetypes . . . 14
5.25 mmap . . . 14
5.26 multiprocessing . . . 14
5.27 os . . . 14
5.28 os.path . . . 15
5.29 pathlib . . . 15
5.30 pdb . . . 16
5.31 queue . . . 16
5.32 random . . . 16
5.33 re . . . 16
5.34 shutil . . . 16
5.35 site . . . 16
5.36 sqlite3 . . . 16
5.37 ssl . . . 16
5.38 statistics . . . 17
5.39 subprocess . . . 17
5.40 sys . . . 17
5.41 tempfile . . . 17
5.42 time . . . 17
5.43 tkinter . . . 17
5.44 traceback . . . 18
5.45 types . . . 18
5.46 typing . . . 18
5.47 unicodedata . . . 18
5.48 venv . . . 18
5.49 warnings . . . 19
5.50 xml . . . 19
5.51 zipimport . . . 19

6 Optimizations **19**

7 Removed Modules And APIs **19**
7.1 PEP 594: Remove “dead batteries” from the standard library . . . 19
7.2 2to3 . . . 21
7.3 builtins . . . 21
7.4 configparser . . . 21
7.5 importlib.metadata . . . 21
7.6 locale . . . 21
7.7 opcode . . . 21
7.8 optparse . . . 22
7.9 pathlib . . . 22
7.10 re . . . 22
7.11 tkinter.tix . . . 22
7.12 turtle . . . 22
7.13 typing . . . 22
7.14 unittest . . . 22
7.15 urllib . . . 23
7.16 webbrowser . . . 23

8 New Deprecations **23**
8.1 Pending Removal in Python 3.14 . . . 25
8.2 Pending Removal in Python 3.15 . . . 26
8.3 Pending removal in Python 3.16 . . . 27
8.4 Pending removal in Python 3.17 . . . 28
8.5 Pending Removal in Future Versions . . . 28

9 CPython Bytecode Changes **31**

10 C API Changes **31**

10.1 New Features . . . 31
10.2 Changed C APIs . . . 33
10.3 Limited C API Changes . . . 34
10.4 Removed C APIs . . . 34
10.5 Deprecated C APIs . . . 36

11 Build Changes **39**

12 Porting to Python 3.13 **39**
12.1 Changes in the Python API . . . 39
12.2 Changes in the C API . . . 40

13 Regression Test Changes **42**

14 Notable changes in 3.13.1 **42**
14.1 sys . . . 42

15 Notable changes in 3.13.4 **42**
15.1 os.path . . . 42
15.2 tarfile . . . 42

Index **43**

Editors
Adam Turner and Thomas Wouters

This article explains the new features in Python 3.13, compared to 3.12. Python 3.13 was released on October 7, 2024. For full details, see the changelog.

See also

PEP 719 – Python 3.13 Release Schedule

1 Summary – Release Highlights

Python 3.13 is the latest stable release of the Python programming language, with a mix of changes to the language, the implementation and the standard library. The biggest changes include a new *interactive interpreter*, experimental support for running in a *free-threaded mode* (**PEP 703**), and a *Just-In-Time compiler* (**PEP 744**).

Error messages continue to improve, with tracebacks now highlighted in color by default. The `locals()` builtin now has *defined semantics* for changing the returned mapping, and type parameters now support default values.

The library changes contain removal of deprecated APIs and modules, as well as the usual improvements in user-friendliness and correctness. Several legacy standard library modules have now *been removed* following their deprecation in Python 3.11 (**PEP 594**).

This article doesn't attempt to provide a complete specification of all new features, but instead gives a convenient overview. For full details refer to the documentation, such as the Library Reference and Language Reference. To understand the complete implementation and design rationale for a change, refer to the PEP for a particular new feature; but note that PEPs usually are not kept up-to-date once a feature has been fully implemented. See *Porting to Python 3.13* for guidance on upgrading from earlier versions of Python.

Interpreter improvements:

- A greatly improved *interactive interpreter* and *improved error messages*.

- **PEP 667**: The `locals()` builtin now has *defined semantics* when mutating the returned mapping. Python debuggers and similar tools may now more reliably update local variables in optimized scopes even during concurrent code execution.
- **PEP 703**: CPython 3.13 has experimental support for running with the global interpreter lock disabled. See *Free-threaded CPython* for more details.
- **PEP 744**: A basic *JIT compiler* was added. It is currently disabled by default (though we may turn it on later). Performance improvements are modest – we expect to improve this over the next few releases.
- Color support in the new *interactive interpreter*, as well as in *tracebacks* and *doctest* output. This can be disabled through the `PYTHON_COLORS` and `NO_COLOR` environment variables.

Python data model improvements:

- `__static_attributes__` stores the names of attributes accessed through `self.X` in any function in a class body.
- `__firstlineno__` records the first line number of a class definition.

Significant improvements in the standard library:

- Add a new `PythonFinalizationError` exception, raised when an operation is blocked during finalization.
- The `argparse` module now supports deprecating command-line options, positional arguments, and subcommands.
- The new functions `base64.z85encode()` and `base64.z85decode()` support encoding and decoding Z85 data.
- The `copy` module now has a `copy.replace()` function, with support for many builtin types and any class defining the `__replace__()` method.
- The new `dbm.sqlite3` module is now the default `dbm` backend.
- The `os` module has a suite of new functions for working with Linux's timer notification file descriptors.
- The `random` module now has a command-line interface.

Security improvements:

- `ssl.create_default_context()` sets `ssl.VERIFY_X509_PARTIAL_CHAIN` and `ssl.VERIFY_X509_STRICT` as default flags.

C API improvements:

- The `Py_mod_gil` slot is now used to indicate that an extension module supports running with the GIL disabled.
- The PyTime C API has been added, providing access to system clocks.
- `PyMutex` is a new lightweight mutex that occupies a single byte.
- There is a new suite of functions for generating **PEP 669** monitoring events in the C API.

New typing features:

- **PEP 696**: Type parameters (`typing.TypeVar`, `typing.ParamSpec`, and `typing.TypeVarTuple`) now support defaults.
- **PEP 702**: The new `warnings.deprecated()` decorator adds support for marking deprecations in the type system and at runtime.
- **PEP 705**: `typing.ReadOnly` can be used to mark an item of a `typing.TypedDict` as read-only for type checkers.
- **PEP 742**: `typing.TypeIs` provides more intuitive type narrowing behavior, as an alternative to `typing.TypeGuard`.

Platform support:

- **PEP 730**: Apple's iOS is now an *officially supported platform*, at **tier 3**.
- **PEP 738**: Android is now an *officially supported platform*, at **tier 3**.

- `wasm32-wasi` is now supported as a **tier 2** platform.
- `wasm32-emscripten` is no longer an officially supported platform.

Important removals:

- *PEP 594*: The remaining 19 "dead batteries" (legacy stdlib modules) have been removed from the standard library: `aifc`, `audioop`, `cgi`, `cgitb`, `chunk`, `crypt`, `imghdr`, `mailcap`, `msilib`, `nis`, `nntplib`, `ossaudiodev`, `pipes`, `sndhdr`, `spwd`, `sunau`, `telnetlib`, `uu` and `xdrlib`.
- Remove the **`2to3`** tool and `lib2to3` module (deprecated in Python 3.11).
- Remove the `tkinter.tix` module (deprecated in Python 3.6).
- Remove the `locale.resetlocale()` function.
- Remove the `typing.io` and `typing.re` namespaces.
- Remove chained `classmethod` descriptors.

Release schedule changes:

PEP 602 ("Annual Release Cycle for Python") has been updated to extend the full support ('bugfix') period for new releases to two years. This updated policy means that:

- Python 3.9–3.12 have one and a half years of full support, followed by three and a half years of security fixes.
- Python 3.13 and later have two years of full support, followed by three years of security fixes.

2 New Features

2.1 A better interactive interpreter

Python now uses a new interactive shell by default, based on code from the PyPy project. When the user starts the REPL from an interactive terminal, the following new features are now supported:

- Multiline editing with history preservation.
- Direct support for REPL-specific commands like `help`, `exit`, and `quit`, without the need to call them as functions.
- Prompts and tracebacks with color enabled by default.
- Interactive help browsing using `F1` with a separate command history.
- History browsing using `F2` that skips output as well as the >>> and … prompts.
- "Paste mode" with `F3` that makes pasting larger blocks of code easier (press `F3` again to return to the regular prompt).

To disable the new interactive shell, set the `PYTHON_BASIC_REPL` environment variable. For more on interactive mode, see tut-interac.

(Contributed by Pablo Galindo Salgado, Łukasz Langa, and Lysandros Nikolaou in gh-111201 based on code from the PyPy project. Windows support contributed by Dino Viehland and Anthony Shaw.)

2.2 Improved error messages

- The interpreter now uses color by default when displaying tracebacks in the terminal. This feature can be controlled via the new `PYTHON_COLORS` environment variable as well as the canonical `NO_COLOR` and `FORCE_COLOR` environment variables. (Contributed by Pablo Galindo Salgado in gh-112730.)
- A common mistake is to write a script with the same name as a standard library module. When this results in errors, we now display a more helpful error message:

```
$ python random.py
Traceback (most recent call last):
  File "/home/me/random.py", line 1, in <module>
    import random
  File "/home/me/random.py", line 3, in <module>
    print(random.randint(5))
          ^^^^^^^^^^^^^^
AttributeError: module 'random' has no attribute 'randint' (consider renaming
↪'/home/me/random.py' since it has the same name as the standard library␣
↪module named 'random' and prevents importing that standard library module)
```

Similarly, if a script has the same name as a third-party module that it attempts to import and this results in errors, we also display a more helpful error message:

```
$ python numpy.py
Traceback (most recent call last):
  File "/home/me/numpy.py", line 1, in <module>
    import numpy as np
  File "/home/me/numpy.py", line 3, in <module>
    np.array([1, 2, 3])
    ^^^^^^^^
AttributeError: module 'numpy' has no attribute 'array' (consider renaming '/
↪home/me/numpy.py' if it has the same name as a library you intended to␣
↪import)
```

(Contributed by Shantanu Jain in gh-95754.)

- The error message now tries to suggest the correct keyword argument when an incorrect keyword argument is passed to a function.

```
>>> "Better error messages!".split(max_split=1)
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    "Better error messages!".split(max_split=1)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^
TypeError: split() got an unexpected keyword argument 'max_split'. Did you␣
↪mean 'maxsplit'?
```

(Contributed by Pablo Galindo Salgado and Shantanu Jain in gh-107944.)

2.3 Free-threaded CPython

CPython now has experimental support for running in a free-threaded mode, with the global interpreter lock (GIL) disabled. This is an experimental feature and therefore is not enabled by default. The free-threaded mode requires a different executable, usually called `python3.13t` or `python3.13t.exe`. Pre-built binaries marked as *free-threaded* can be installed as part of the official Windows and macOS installers, or CPython can be built from source with the `--disable-gil` option.

Free-threaded execution allows for full utilization of the available processing power by running threads in parallel on available CPU cores. While not all software will benefit from this automatically, programs designed with threading in mind will run faster on multi-core hardware. **The free-threaded mode is experimental** and work is ongoing to improve it: expect some bugs and a substantial single-threaded performance hit. Free-threaded builds of CPython support optionally running with the GIL enabled at runtime using the environment variable `PYTHON_GIL` or the command-line option `-X gil=1`.

To check if the current interpreter supports free-threading, `python -VV` and `sys.version` contain "experimental free-threading build". The new `sys._is_gil_enabled()` function can be used to check whether the GIL is actually disabled in the running process.

C-API extension modules need to be built specifically for the free-threaded build. Extensions that support running with the GIL disabled should use the `Py_mod_gil` slot. Extensions using single-phase init should use `PyUnstable_Module_SetGIL()` to indicate whether they support running with the GIL disabled. Importing C extensions that don't use these mechanisms will cause the GIL to be enabled, unless the GIL was explicitly disabled with the `PYTHON_GIL` environment variable or the `-X gil=0` option. pip 24.1 or newer is required to install packages with C extensions in the free-threaded build.

This work was made possible thanks to many individuals and organizations, including the large community of contributors to Python and third-party projects to test and enable free-threading support. Notable contributors include: Sam Gross, Ken Jin, Donghee Na, Itamar Oren, Matt Page, Brett Simmers, Dino Viehland, Carl Meyer, Nathan Goldbaum, Ralf Gommers, Lysandros Nikolaou, and many others. Many of these contributors are employed by Meta, which has provided significant engineering resources to support this project.

See also

PEP 703 "Making the Global Interpreter Lock Optional in CPython" contains rationale and information surrounding this work.

Porting Extension Modules to Support Free-Threading: A community-maintained porting guide for extension authors.

2.4 An experimental just-in-time (JIT) compiler

When CPython is configured and built using the `--enable-experimental-jit` option, a just-in-time (JIT) compiler is added which may speed up some Python programs. On Windows, use `PCbuild/build.bat --experimental-jit` to enable the JIT or `--experimental-jit-interpreter` to enable the Tier 2 interpreter. Build requirements and further supporting information are contained at `Tools/jit/README.md`.

The `--enable-experimental-jit` option takes these (optional) values, defaulting to `yes` if `--enable-experimental-jit` is present without the optional value.

- `no`: Disable the entire Tier 2 and JIT pipeline.
- `yes`: Enable the JIT. To disable the JIT at runtime, pass the environment variable `PYTHON_JIT=0`.
- `yes-off`: Build the JIT but disable it by default. To enable the JIT at runtime, pass the environment variable `PYTHON_JIT=1`.
- `interpreter`: Enable the Tier 2 interpreter but disable the JIT. The interpreter can be disabled by running with `PYTHON_JIT=0`.

The internal architecture is roughly as follows:

- We start with specialized *Tier 1 bytecode*. See What's new in 3.11 for details.
- When the Tier 1 bytecode gets hot enough, it gets translated to a new purely internal intermediate representation (IR), called the *Tier 2 IR*, and sometimes referred to as micro-ops ("uops").
- The Tier 2 IR uses the same stack-based virtual machine as Tier 1, but the instruction format is better suited to translation to machine code.
- We have several optimization passes for Tier 2 IR, which are applied before it is interpreted or translated to machine code.
- There is a Tier 2 interpreter, but it is mostly intended for debugging the earlier stages of the optimization pipeline. The Tier 2 interpreter can be enabled by configuring Python with `--enable-experimental-jit=interpreter`.
- When the JIT is enabled, the optimized Tier 2 IR is translated to machine code, which is then executed.
- The machine code translation process uses a technique called *copy-and-patch*. It has no runtime dependencies, but there is a new build-time dependency on LLVM.

See also

PEP 744

(JIT by Brandt Bucher, inspired by a paper by Haoran Xu and Fredrik Kjolstad. Tier 2 IR by Mark Shannon and Guido van Rossum. Tier 2 optimizer by Ken Jin.)

2.5 Defined mutation semantics for `locals()`

Historically, the expected result of mutating the return value of `locals()` has been left to individual Python implementations to define. Starting from Python 3.13, **PEP 667** standardises the historical behavior of CPython for most code execution scopes, but changes optimized scopes (functions, generators, coroutines, comprehensions, and generator expressions) to explicitly return independent snapshots of the currently assigned local variables, including locally referenced nonlocal variables captured in closures.

This change to the semantics of `locals()` in optimized scopes also affects the default behavior of code execution functions that implicitly target `locals()` if no explicit namespace is provided (such as `exec()` and `eval()`). In previous versions, whether or not changes could be accessed by calling `locals()` after calling the code execution function was implementation-dependent. In CPython specifically, such code would typically appear to work as desired, but could sometimes fail in optimized scopes based on other code (including debuggers and code execution tracing tools) potentially resetting the shared snapshot in that scope. Now, the code will always run against an independent snapshot of the local variables in optimized scopes, and hence the changes will never be visible in subsequent calls to `locals()`. To access the changes made in these cases, an explicit namespace reference must now be passed to the relevant function. Alternatively, it may make sense to update affected code to use a higher level code execution API that returns the resulting code execution namespace (e.g. `runpy.run_path()` when executing Python files from disk).

To ensure debuggers and similar tools can reliably update local variables in scopes affected by this change, `FrameType.f_locals` now returns a write-through proxy to the frame's local and locally referenced nonlocal variables in these scopes, rather than returning an inconsistently updated shared `dict` instance with undefined runtime semantics.

See **PEP 667** for more details, including related C API changes and deprecations. Porting notes are also provided below for the affected *Python APIs* and *C APIs*.

(PEP and implementation contributed by Mark Shannon and Tian Gao in gh-74929. Documentation updates provided by Guido van Rossum and Alyssa Coghlan.)

2.6 Support for mobile platforms

PEP 730: iOS is now a **PEP 11** supported platform, with the `arm64-apple-ios` and `arm64-apple-ios-simulator` targets at tier 3 (iPhone and iPad devices released after 2013 and the Xcode iOS simulator running on Apple silicon hardware, respectively). `x86_64-apple-ios-simulator` (the Xcode iOS simulator running on older `x86_64` hardware) is not a tier 3 supported platform, but will have best-effort support. (PEP written and implementation contributed by Russell Keith-Magee in gh-114099.)

PEP 738: Android is now a **PEP 11** supported platform, with the `aarch64-linux-android` and `x86_64-linux-android` targets at tier 3. The 32-bit targets `arm-linux-androideabi` and `i686-linux-android` are not tier 3 supported platforms, but will have best-effort support. (PEP written and implementation contributed by Malcolm Smith in gh-116622.)

See also

PEP 730, **PEP 738**

3 Other Language Changes

- The compiler now strips common leading whitespace from every line in a docstring. This reduces the size of the bytecode cache (such as `.pyc` files), with reductions in file size of around 5%, for example in `sqlalchemy.orm.session` from SQLAlchemy 2.0. This change affects tools that use docstrings, such as `doctest`.

```
>>> def spam():
...     """
...         This is a docstring with
...           leading whitespace.
...
...         It even has multiple paragraphs!
...     """
...
>>> spam.__doc__
'\nThis is a docstring with\n  leading whitespace.\n\nIt even has multiple␣
↪paragraphs!\n'
```

  (Contributed by Inada Naoki in gh-81283.)

- Annotation scopes within class scopes can now contain lambdas and comprehensions. Comprehensions that are located within class scopes are not inlined into their parent scope.

```
class C[T]:
    type Alias = lambda: T
```

  (Contributed by Jelle Zijlstra in gh-109118 and gh-118160.)

- Future statements are no longer triggered by relative imports of the `__future__` module, meaning that statements of the form `from .__future__ import ...` are now simply standard relative imports, with no special features activated. (Contributed by Jeremiah Gabriel Pascual in gh-118216.)

- `global` declarations are now permitted in `except` blocks when that global is used in the `else` block. Previously this raised an erroneous `SyntaxError`. (Contributed by Irit Katriel in gh-111123.)

- Add `PYTHON_FROZEN_MODULES`, a new environment variable that determines whether frozen modules are ignored by the import machinery, equivalent to the `-X frozen_modules` command-line option. (Contributed by Yilei Yang in gh-111374.)

- Add support for the perf profiler working without frame pointers through the new environment variable `PYTHON_PERF_JIT_SUPPORT` and command-line option `-X perf_jit`. (Contributed by Pablo Galindo in gh-118518.)

- The location of a `.python_history` file can be changed via the new `PYTHON_HISTORY` environment variable. (Contributed by Levi Sabah, Zackery Spytz and Hugo van Kemenade in gh-73965.)

- Classes have a new `__static_attributes__` attribute. This is populated by the compiler with a tuple of the class's attribute names which are assigned through `self.<name>` from any function in its body. (Contributed by Irit Katriel in gh-115775.)

- The compiler now creates a `__firstlineno__` attribute on classes with the line number of the first line of the class definition. (Contributed by Serhiy Storchaka in gh-118465.)

- The `exec()` and `eval()` builtins now accept the *globals* and *locals* arguments as keywords. (Contributed by Raphael Gaschignard in gh-105879)

- The `compile()` builtin now accepts a new flag, `ast.PyCF_OPTIMIZED_AST`, which is similar to `ast.PyCF_ONLY_AST` except that the returned AST is optimized according to the value of the *optimize* argument. (Contributed by Irit Katriel in gh-108113).

- Add a `__name__` attribute on `property` objects. (Contributed by Eugene Toder in gh-101860.)

- Add `PythonFinalizationError`, a new exception derived from `RuntimeError` and used to signal when operations are blocked during finalization. The following callables now raise `PythonFinalizationError`,

instead of `RuntimeError`:

- `_thread.start_new_thread()`
- `os.fork()`
- `os.forkpty()`
- `subprocess.Popen`

(Contributed by Victor Stinner in gh-114570.)

- Allow the *count* argument of `str.replace()` to be a keyword. (Contributed by Hugo van Kemenade in gh-106487.)
- Many functions now emit a warning if a boolean value is passed as a file descriptor argument. This can help catch some errors earlier. (Contributed by Serhiy Storchaka in gh-82626.)
- Added `name` and `mode` attributes for compressed and archived file-like objects in the `bz2`, `lzma`, `tarfile`, and `zipfile` modules. (Contributed by Serhiy Storchaka in gh-115961.)

4 New Modules

- `dbm.sqlite3`: An SQLite backend for `dbm`. (Contributed by Raymond Hettinger and Erlend E. Aasland in gh-100414.)

5 Improved Modules

5.1 argparse

- Add the *deprecated* parameter to the `add_argument()` and `add_parser()` methods, to enable deprecating command-line options, positional arguments, and subcommands. (Contributed by Serhiy Storchaka in gh-83648.)

5.2 array

- Add the `'w'` type code (`Py_UCS4`) for Unicode characters. It should be used instead of the deprecated `'u'` type code. (Contributed by Inada Naoki in gh-80480.)
- Register `array.array` as a `MutableSequence` by implementing the `clear()` method. (Contributed by Mike Zimin in gh-114894.)

5.3 ast

- The constructors of node types in the `ast` module are now stricter in the arguments they accept, with more intuitive behavior when arguments are omitted.

  If an optional field on an AST node is not included as an argument when constructing an instance, the field will now be set to `None`. Similarly, if a list field is omitted, that field will now be set to an empty list, and if an `expr_context` field is omitted, it defaults to `Load()`. (Previously, in all cases, the attribute would be missing on the newly constructed AST node instance.)

  In all other cases, where a required argument is omitted, the node constructor will emit a `DeprecationWarning`. This will raise an exception in Python 3.15. Similarly, passing a keyword argument to the constructor that does not map to a field on the AST node is now deprecated, and will raise an exception in Python 3.15.

  These changes do not apply to user-defined subclasses of `ast.AST` unless the class opts in to the new behavior by defining the `AST._field_types` mapping.

  (Contributed by Jelle Zijlstra in gh-105858, gh-117486, and gh-118851.)

- `ast.parse()` now accepts an optional argument *optimize* which is passed on to `compile()`. This makes it possible to obtain an optimized AST. (Contributed by Irit Katriel in gh-108113.)

5.4 asyncio

- `asyncio.as_completed()` now returns an object that is both an asynchronous iterator and a plain iterator of awaitables. The awaitables yielded by asynchronous iteration include original task or future objects that were passed in, making it easier to associate results with the tasks being completed. (Contributed by Justin Arthur in gh-77714.)
- `asyncio.loop.create_unix_server()` will now automatically remove the Unix socket when the server is closed. (Contributed by Pierre Ossman in gh-111246.)
- `DatagramTransport.sendto()` will now send zero-length datagrams if called with an empty bytes object. The transport flow control also now accounts for the datagram header when calculating the buffer size. (Contributed by Jamie Phan in gh-115199.)
- Add `Queue.shutdown` and `QueueShutDown` to manage queue termination. (Contributed by Laurie Opperman and Yves Duprat in gh-104228.)
- Add the `Server.close_clients()` and `Server.abort_clients()` methods, which more forcefully close an asyncio server. (Contributed by Pierre Ossman in gh-113538.)
- Accept a tuple of separators in `StreamReader.readuntil()`, stopping when any one of them is encountered. (Contributed by Bruce Merry in gh-81322.)
- Improve the behavior of `TaskGroup` when an external cancellation collides with an internal cancellation. For example, when two task groups are nested and both experience an exception in a child task simultaneously, it was possible that the outer task group would hang, because its internal cancellation was swallowed by the inner task group.

  In the case where a task group is cancelled externally and also must raise an `ExceptionGroup`, it will now call the parent task's `cancel()` method. This ensures that a `CancelledError` will be raised at the next `await`, so the cancellation is not lost.

  An added benefit of these changes is that task groups now preserve the cancellation count (`cancelling()`).

  In order to handle some corner cases, `uncancel()` may now reset the undocumented `_must_cancel` flag when the cancellation count reaches zero.

  (Inspired by an issue reported by Arthur Tacca in gh-116720.)
- When `TaskGroup.create_task()` is called on an inactive `TaskGroup`, the given coroutine will be closed (which prevents a `RuntimeWarning` about the given coroutine being never awaited). (Contributed by Arthur Tacca and Jason Zhang in gh-115957.)
- The function and methods named `create_task` have received a new `**kwargs` argument that is passed through to the task constructor. This change was accidentally added in 3.13.3, and broke the API contract for custom task factories. Several third-party task factories implemented workarounds for this. In 3.13.4 and later releases the old factory contract is honored once again (until 3.14). To keep the workarounds working, the extra `**kwargs` argument still allows passing additional keyword arguments to `Task` and to custom task factories.

  This affects the following function and methods: `asyncio.create_task()`, `asyncio.loop.create_task()`, `asyncio.TaskGroup.create_task()`. (Contributed by Thomas Grainger in gh-128307.)

5.5 base64

- Add `z85encode()` and `z85decode()` functions for encoding `bytes` as Z85 data and decoding Z85-encoded data to `bytes`. (Contributed by Matan Perelman in gh-75299.)

5.6 compileall

- The default number of worker threads and processes is now selected using `os.process_cpu_count()` instead of `os.cpu_count()`. (Contributed by Victor Stinner in gh-109649.)

5.7 concurrent.futures

- The default number of worker threads and processes is now selected using `os.process_cpu_count()` instead of `os.cpu_count()`. (Contributed by Victor Stinner in gh-109649.)

5.8 configparser

- `ConfigParser` now has support for unnamed sections, which allows for top-level key-value pairs. This can be enabled with the new *allow_unnamed_section* parameter. (Contributed by Pedro Sousa Lacerda in gh-66449.)

5.9 copy

- The new `replace()` function and the `replace protocol` make creating modified copies of objects much simpler. This is especially useful when working with immutable objects. The following types support the `replace()` function and implement the replace protocol:
  - `collections.namedtuple()`
  - `dataclasses.dataclass`
  - `datetime.datetime`, `datetime.date`, `datetime.time`
  - `inspect.Signature`, `inspect.Parameter`
  - `types.SimpleNamespace`
  - code objects

  Any user-defined class can also support `copy.replace()` by defining the `__replace__()` method. (Contributed by Serhiy Storchaka in gh-108751.)

5.10 ctypes

- As a consequence of necessary internal refactoring, initialization of internal metaclasses now happens in `__init__` rather than in `__new__`. This affects projects that subclass these internal metaclasses to provide custom initialization. Generally:
  - Custom logic that was done in `__new__` after calling `super().__new__` should be moved to `__init__`.
  - To create a class, call the metaclass, not only the metaclass's `__new__` method.

  See gh-124520 for discussion and links to changes in some affected projects.
- `ctypes.Structure` objects have a new `_align_` attribute which allows the alignment of the structure being packed to/from memory to be specified explicitly. (Contributed by Matt Sanderson in gh-112433)

5.11 dbm

- Add `dbm.sqlite3`, a new module which implements an SQLite backend, and make it the default `dbm` backend. (Contributed by Raymond Hettinger and Erlend E. Aasland in gh-100414.)
- Allow removing all items from the database through the new `clear()` methods of the GDBM and NDBM database objects. (Contributed by Donghee Na in gh-107122.)

5.12 dis

- Change the output of `dis` module functions to show logical labels for jump targets and exception handlers, rather than offsets. The offsets can be added with the new `-O` command-line option or the *show_offsets* argument. (Contributed by Irit Katriel in gh-112137.)
- `get_instructions()` no longer represents cache entries as separate instructions. Instead, it returns them as part of the `Instruction`, in the new *cache_info* field. The *show_caches* argument to `get_instructions()` is deprecated and no longer has any effect. (Contributed by Irit Katriel in gh-112962.)

5.13 doctest

- `doctest` output is now colored by default. This can be controlled via the new `PYTHON_COLORS` environment variable as well as the canonical `NO_COLOR` and `FORCE_COLOR` environment variables. See also using-on-controlling-color. (Contributed by Hugo van Kemenade in gh-117225.)
- The `DocTestRunner.run()` method now counts the number of skipped tests. Add the `DocTestRunner.skips` and `TestResults.skipped` attributes. (Contributed by Victor Stinner in gh-108794.)

5.14 email

- Headers with embedded newlines are now quoted on output. The `generator` will now refuse to serialize (write) headers that are improperly folded or delimited, such that they would be parsed as multiple headers or joined with adjacent data. If you need to turn this safety feature off, set `verify_generated_headers`. (Contributed by Bas Bloemsaat and Petr Viktorin in gh-121650.)
- `getaddresses()` and `parseaddr()` now return `('', '')` pairs in more situations where invalid email addresses are encountered instead of potentially inaccurate values. The two functions have a new optional *strict* parameter (default `True`). To get the old behavior (accepting malformed input), use `strict=False`. `getattr(email.utils, 'supports_strict_parsing', False)` can be used to check if the *strict* parameter is available. (Contributed by Thomas Dwyer and Victor Stinner for gh-102988 to improve the **CVE 2023-27043** fix.)

5.15 enum

- `EnumDict` has been made public to better support subclassing `EnumType`.

5.16 fractions

- `Fraction` objects now support the standard format specification mini-language rules for fill, alignment, sign handling, minimum width, and grouping. (Contributed by Mark Dickinson in gh-111320.)

5.17 glob

- Add `translate()`, a function to convert a path specification with shell-style wildcards to a regular expression. (Contributed by Barney Gale in gh-72904.)

5.18 importlib

- The following functions in `importlib.resources` now allow accessing a directory (or tree) of resources, using multiple positional arguments (the *encoding* and *errors* arguments in the text-reading functions are now keyword-only):
  - `is_resource()`
  - `open_binary()`
  - `open_text()`
  - `path()`
  - `read_binary()`
  - `read_text()`

  These functions are no longer deprecated and are not scheduled for removal. (Contributed by Petr Viktorin in gh-116608.)
- `contents()` remains deprecated in favor of the fully-featured `Traversable` API. However, there is now no plan to remove it. (Contributed by Petr Viktorin in gh-116608.)

5.19 io

- The `IOBase` finalizer now logs any errors raised by the `close()` method with `sys.unraisablehook`. Previously, errors were ignored silently by default, and only logged in Python Development Mode or when using a Python debug build. (Contributed by Victor Stinner in gh-62948.)

5.20 ipaddress

- Add the `IPv4Address.ipv6_mapped` property, which returns the IPv4-mapped IPv6 address. (Contributed by Charles Machalow in gh-109466.)
- Fix `is_global` and `is_private` behavior in `IPv4Address`, `IPv6Address`, `IPv4Network`, and `IPv6Network`. (Contributed by Jakub Stasiak in gh-113171.)

5.21 itertools

- `batched()` has a new *strict* parameter, which raises a `ValueError` if the final batch is shorter than the specified batch size. (Contributed by Raymond Hettinger in gh-113202.)

5.22 marshal

- Add the *allow_code* parameter in module functions. Passing `allow_code=False` prevents serialization and de-serialization of code objects which are incompatible between Python versions. (Contributed by Serhiy Storchaka in gh-113626.)

5.23 math

- The new function `fma()` performs fused multiply-add operations. This computes `x * y + z` with only a single round, and so avoids any intermediate loss of precision. It wraps the `fma()` function provided by C99, and follows the specification of the IEEE 754 "fusedMultiplyAdd" operation for special cases. (Contributed by Mark Dickinson and Victor Stinner in gh-73468.)

5.24 mimetypes

- Add the `guess_file_type()` function to guess a MIME type from a filesystem path. Using paths with `guess_type()` is now soft deprecated. (Contributed by Serhiy Storchaka in gh-66543.)

5.25 mmap

- `mmap` is now protected from crashing on Windows when the mapped memory is inaccessible due to file system errors or access violations. (Contributed by Jannis Weigend in gh-118209.)
- `mmap` has a new `seekable()` method that can be used when a seekable file-like object is required. The `seek()` method now returns the new absolute position. (Contributed by Donghee Na and Sylvie Liberman in gh-111835.)
- The new UNIX-only *trackfd* parameter for `mmap` controls file descriptor duplication; if false, the file descriptor specified by *fileno* will not be duplicated. (Contributed by Zackery Spytz and Petr Viktorin in gh-78502.)

5.26 multiprocessing

- The default number of worker threads and processes is now selected using `os.process_cpu_count()` instead of `os.cpu_count()`. (Contributed by Victor Stinner in gh-109649.)

5.27 os

- Add `process_cpu_count()` function to get the number of logical CPU cores usable by the calling thread of the current process. (Contributed by Victor Stinner in gh-109649.)

- `cpu_count()` and `process_cpu_count()` can be overridden through the new environment variable `PYTHON_CPU_COUNT` or the new command-line option `-X cpu_count`. This option is useful for users who need to limit CPU resources of a container system without having to modify application code or the container itself. (Contributed by Donghee Na in gh-109595.)
- Add a low level interface to Linux's *timer file descriptors* via `timerfd_create()`, `timerfd_settime()`, `timerfd_settime_ns()`, `timerfd_gettime()`, `timerfd_gettime_ns()`, `TFD_NONBLOCK`, `TFD_CLOEXEC`, `TFD_TIMER_ABSTIME`, and `TFD_TIMER_CANCEL_ON_SET` (Contributed by Masaru Tsuchiyama in gh-108277.)
- `lchmod()` and the *follow_symlinks* argument of `chmod()` are both now available on Windows. Note that the default value of *follow_symlinks* in `lchmod()` is `False` on Windows. (Contributed by Serhiy Storchaka in gh-59616.)
- `fchmod()` and support for file descriptors in `chmod()` are both now available on Windows. (Contributed by Serhiy Storchaka in gh-113191.)
- On Windows, `mkdir()` and `makedirs()` now support passing a *mode* value of `0o700` to apply access control to the new directory. This implicitly affects `tempfile.mkdtemp()` and is a mitigation for **CVE 2024-4030**. Other values for *mode* continue to be ignored. (Contributed by Steve Dower in gh-118486.)
- `posix_spawn()` now accepts `None` for the *env* argument, which makes the newly spawned process use the current process environment. (Contributed by Jakub Kulik in gh-113119.)
- `posix_spawn()` can now use the `POSIX_SPAWN_CLOSEFROM` attribute in the *file_actions* parameter on platforms that support `posix_spawn_file_actions_addclosefrom_np()`. (Contributed by Jakub Kulik in gh-113117.)

5.28 os.path

- Add `isreserved()` to check if a path is reserved on the current system. This function is only available on Windows. (Contributed by Barney Gale in gh-88569.)
- On Windows, `isabs()` no longer considers paths starting with exactly one slash (`\` or `/`) to be absolute. (Contributed by Barney Gale and Jon Foster in gh-44626.)
- `realpath()` now resolves MS-DOS style file names even if the file is not accessible. (Contributed by Moonsik Park in gh-82367.)

5.29 pathlib

- Add `UnsupportedOperation`, which is raised instead of `NotImplementedError` when a path operation isn't supported. (Contributed by Barney Gale in gh-89812.)
- Add a new constructor for creating `Path` objects from 'file' URIs (`file:///`), `Path.from_uri()`. (Contributed by Barney Gale in gh-107465.)
- Add `PurePath.full_match()` for matching paths with shell-style wildcards, including the recursive wildcard "`**`". (Contributed by Barney Gale in gh-73435.)
- Add the `PurePath.parser` class attribute to store the implementation of `os.path` used for low-level path parsing and joining. This will be either `posixpath` or `ntpath`.
- Add *recurse_symlinks* keyword-only argument to `Path.glob()` and `rglob()`. (Contributed by Barney Gale in gh-77609.)
- `Path.glob()` and `rglob()` now return files and directories when given a pattern that ends with "`**`". Previously, only directories were returned. (Contributed by Barney Gale in gh-70303.)
- Add the *follow_symlinks* keyword-only argument to `Path.is_file`, `Path.is_dir`, `Path.owner()`, and `Path.group()`. (Contributed by Barney Gale in gh-105793 and Kamil Turek in gh-107962.)

5.30 pdb

- `breakpoint()` and `set_trace()` now enter the debugger immediately rather than on the next line of code to be executed. This change prevents the debugger from breaking outside of the context when `breakpoint()` is positioned at the end of the context. (Contributed by Tian Gao in gh-118579.)
- `sys.path[0]` is no longer replaced by the directory of the script being debugged when `sys.flags.safe_path` is set. (Contributed by Tian Gao and Christian Walther in gh-111762.)
- `zipapp` is now supported as a debugging target. (Contributed by Tian Gao in gh-118501.)
- Add ability to move between chained exceptions during post-mortem debugging in `pm()` using the new `exceptions [exc_number]` command for Pdb. (Contributed by Matthias Bussonnier in gh-106676.)
- Expressions and statements whose prefix is a pdb command are now correctly identified and executed. (Contributed by Tian Gao in gh-108464.)

5.31 queue

- Add `Queue.shutdown` and `ShutDown` to manage queue termination. (Contributed by Laurie Opperman and Yves Duprat in gh-104750.)

5.32 random

- Add a command-line interface. (Contributed by Hugo van Kemenade in gh-118131.)

5.33 re

- Rename `re.error` to `PatternError` for improved clarity. `re.error` is kept for backward compatibility.

5.34 shutil

- Support the *dir_fd* and *follow_symlinks* keyword arguments in `chown()`. (Contributed by Berker Peksag and Tahia K in gh-62308)

5.35 site

- `.pth` files are now decoded using UTF-8 first, and then with the locale encoding if UTF-8 decoding fails. (Contributed by Inada Naoki in gh-117802.)

5.36 sqlite3

- A `ResourceWarning` is now emitted if a `Connection` object is not `closed` explicitly. (Contributed by Erlend E. Aasland in gh-105539.)
- Add the *filter* keyword-only parameter to `Connection.iterdump()` for filtering database objects to dump. (Contributed by Mariusz Felisiak in gh-91602.)

5.37 ssl

- The `create_default_context()` API now includes `VERIFY_X509_PARTIAL_CHAIN` and `VERIFY_X509_STRICT` in its default flags.

> **Note**
>
> `VERIFY_X509_STRICT` may reject pre-**RFC 5280** or malformed certificates that the underlying OpenSSL implementation might otherwise accept. Whilst disabling this is not recommended, you can do so using:

```
import ssl

ctx = ssl.create_default_context()
ctx.verify_flags &= ~ssl.VERIFY_X509_STRICT
```

(Contributed by William Woodruff in gh-112389.)

5.38 statistics

- Add `kde()` for kernel density estimation. This makes it possible to estimate a continuous probability density function from a fixed number of discrete samples. (Contributed by Raymond Hettinger in gh-115863.)
- Add `kde_random()` for sampling from an estimated probability density function created by `kde()`. (Contributed by Raymond Hettinger in gh-115863.)

5.39 subprocess

- The `subprocess` module now uses the `posix_spawn()` function in more situations.

  Notably, when *close_fds* is `True` (the default), `posix_spawn()` will be used when the C library provides `posix_spawn_file_actions_addclosefrom_np()`, which includes recent versions of Linux, FreeBSD, and Solaris. On Linux, this should perform similarly to the existing Linux `vfork()` based code.

  A private control knob `subprocess._USE_POSIX_SPAWN` can be set to `False` if you need to force `subprocess` to never use `posix_spawn()`. Please report your reason and platform details in the issue tracker if you set this so that we can improve our API selection logic for everyone. (Contributed by Jakub Kulik in gh-113117.)

5.40 sys

- Add the `_is_interned()` function to test if a string was interned. This function is not guaranteed to exist in all implementations of Python. (Contributed by Serhiy Storchaka in gh-78573.)

5.41 tempfile

- On Windows, the default mode `0o700` used by `tempfile.mkdtemp()` now limits access to the new directory due to changes to `os.mkdir()`. This is a mitigation for **CVE 2024-4030**. (Contributed by Steve Dower in gh-118486.)

5.42 time

- On Windows, `monotonic()` now uses the `QueryPerformanceCounter()` clock for a resolution of 1 microsecond, instead of the `GetTickCount64()` clock which has a resolution of 15.6 milliseconds. (Contributed by Victor Stinner in gh-88494.)
- On Windows, `time()` now uses the `GetSystemTimePreciseAsFileTime()` clock for a resolution of 1 microsecond, instead of the `GetSystemTimeAsFileTime()` clock which has a resolution of 15.6 milliseconds. (Contributed by Victor Stinner in gh-63207.)

5.43 tkinter

- Add `tkinter` widget methods: `tk_busy_hold()`, `tk_busy_configure()`, `tk_busy_cget()`, `tk_busy_forget()`, `tk_busy_current()`, and `tk_busy_status()`. (Contributed by Miguel, klappnase and Serhiy Storchaka in gh-72684.)
- The `tkinter` widget method `wm_attributes()` now accepts the attribute name without the minus prefix to get window attributes, for example `w.wm_attributes('alpha')` and allows specifying attributes and values to set as keyword arguments, for example `w.wm_attributes(alpha=0.5)`. (Contributed by Serhiy Storchaka in gh-43457.)
- `wm_attributes()` can now return attributes as a `dict`, by using the new optional keyword-only parameter *return_python_dict*. (Contributed by Serhiy Storchaka in gh-43457.)

- `Text.count()` can now return a simple `int` when the new optional keyword-only parameter *return_ints* is used. Otherwise, the single count is returned as a 1-tuple or `None`. (Contributed by Serhiy Storchaka in gh-97928.)
- Support the "vsapi" element type in the `element_create()` method of `tkinter.ttk.Style`. (Contributed by Serhiy Storchaka in gh-68166.)
- Add the `after_info()` method for Tkinter widgets. (Contributed by Cheryl Sabella in gh-77020.)
- Add a new `copy_replace()` method to `PhotoImage` to copy a region from one image to another, possibly with pixel zooming, subsampling, or both. (Contributed by Serhiy Storchaka in gh-118225.)
- Add *from_coords* parameter to the `PhotoImage` methods `copy()`, `zoom()` and `subsample()`. Add *zoom* and *subsample* parameters to the `PhotoImage` method `copy()`. (Contributed by Serhiy Storchaka in gh-118225.)
- Add the `PhotoImage` methods `read()` to read an image from a file and `data()` to get the image data. Add *background* and *grayscale* parameters to the `write()` method. (Contributed by Serhiy Storchaka in gh-118271.)

5.44 traceback

- Add the `exc_type_str` attribute to `TracebackException`, which holds a string display of the *exc_type*. Deprecate the `exc_type` attribute, which holds the type object itself. Add parameter *save_exc_type* (default `True`) to indicate whether `exc_type` should be saved. (Contributed by Irit Katriel in gh-112332.)
- Add a new *show_group* keyword-only parameter to `TracebackException.format_exception_only()` to (recursively) format the nested exceptions of a `BaseExceptionGroup` instance. (Contributed by Irit Katriel in gh-105292.)

5.45 types

- `SimpleNamespace` can now take a single positional argument to initialise the namespace's arguments. This argument must either be a mapping or an iterable of key-value pairs. (Contributed by Serhiy Storchaka in gh-108191.)

5.46 typing

- **PEP 705**: Add `ReadOnly`, a special typing construct to mark a `TypedDict` item as read-only for type checkers.
- **PEP 742**: Add `TypeIs`, a typing construct that can be used to instruct a type checker how to narrow a type.
- Add `NoDefault`, a sentinel object used to represent the defaults of some parameters in the `typing` module. (Contributed by Jelle Zijlstra in gh-116126.)
- Add `get_protocol_members()` to return the set of members defining a `typing.Protocol`. (Contributed by Jelle Zijlstra in gh-104873.)
- Add `is_protocol()` to check whether a class is a `Protocol`. (Contributed by Jelle Zijlstra in gh-104873.)
- `ClassVar` can now be nested in `Final`, and vice versa. (Contributed by Mehdi Drissi in gh-89547.)

5.47 unicodedata

- Update the Unicode database to version 15.1.0. (Contributed by James Gerity in gh-109559.)

5.48 venv

- Add support for creating source control management (SCM) ignore files in a virtual environment's directory. By default, Git is supported. This is implemented as opt-in via the API, which can be extended to support other SCMs (`EnvBuilder` and `create()`), and opt-out via the CLI, using `--without-scm-ignore-files`. (Contributed by Brett Cannon in gh-108125.)

5.49 warnings

- **PEP 702**: The new `warnings.deprecated()` decorator provides a way to communicate deprecations to a static type checker and to warn on usage of deprecated classes and functions. A `DeprecationWarning` may also be emitted when a decorated function or class is used at runtime. (Contributed by Jelle Zijlstra in gh-104003.)

5.50 xml

- Allow controlling Expat >=2.6.0 reparse deferral (**CVE 2023-52425**) by adding five new methods:
  - `xml.etree.ElementTree.XMLParser.flush()`
  - `xml.etree.ElementTree.XMLPullParser.flush()`
  - `xml.parsers.expat.xmlparser.GetReparseDeferralEnabled()`
  - `xml.parsers.expat.xmlparser.SetReparseDeferralEnabled()`
  - `xml.sax.expatreader.ExpatParser.flush()`

  (Contributed by Sebastian Pipping in gh-115623.)
- Add the `close()` method for the iterator returned by `iterparse()` for explicit cleanup. (Contributed by Serhiy Storchaka in gh-69893.)

5.51 zipimport

- Add support for ZIP64 format files. Everybody loves huge data, right? (Contributed by Tim Hatch in gh-94146.)

6 Optimizations

- Several standard library modules have had their import times significantly improved. For example, the import time of the `typing` module has been reduced by around a third by removing dependencies on `re` and `contextlib`. Other modules to enjoy import-time speedups include `email.utils`, `enum`, `functools`, `importlib.metadata`, and `threading`. (Contributed by Alex Waygood, Shantanu Jain, Adam Turner, Daniel Hollas, and others in gh-109653.)
- `textwrap.indent()` is now around 30% faster than before for large input. (Contributed by Inada Naoki in gh-107369.)
- The `subprocess` module now uses the `posix_spawn()` function in more situations, including when *close_fds* is `True` (the default) on many modern platforms. This should provide a notable performance increase when launching processes on FreeBSD and Solaris. See the *subprocess* section above for details. (Contributed by Jakub Kulik in gh-113117.)

7 Removed Modules And APIs

7.1 PEP 594: Remove “dead batteries” from the standard library

PEP 594 proposed removing 19 modules from the standard library, colloquially referred to as ‘dead batteries’ due to their historic, obsolete, or insecure status. All of the following modules were deprecated in Python 3.11, and are now removed:

- `aifc`
  - standard-aifc: Use the redistribution of `aifc` library from PyPI.
- `audioop`
  - audioop-lts: Use `audioop-lts` library from PyPI.
- `chunk`

  - standard-chunk: Use the redistribution of `chunk` library from PyPI.

- `cgi` and `cgitb`
  - `cgi.FieldStorage` can typically be replaced with `urllib.parse.parse_qsl()` for `GET` and `HEAD` requests, and the `email.message` module or the multipart library for `POST` and `PUT` requests.
  - `cgi.parse()` can be replaced by calling `urllib.parse.parse_qs()` directly on the desired query string, unless the input is `multipart/form-data`, which should be replaced as described below for `cgi.parse_multipart()`.
  - `cgi.parse_header()` can be replaced with the functionality in the `email` package, which implements the same MIME RFCs. For example, with `email.message.EmailMessage`:

```
from email.message import EmailMessage

msg = EmailMessage()
msg['content-type'] = 'application/json; charset="utf8"'
main, params = msg.get_content_type(), msg['content-type'].params
```

  - `cgi.parse_multipart()` can be replaced with the functionality in the `email` package, which implements the same MIME RFCs, or with the multipart library. For example, the `email.message.EmailMessage` and `email.message.Message` classes.
  - standard-cgi: and standard-cgitb: Use the redistribution of `cgi` and `cgitb` library from PyPI.

- `crypt` and the private `_crypt` extension. The `hashlib` module may be an appropriate replacement when simply hashing a value is required. Otherwise, various third-party libraries on PyPI are available:
  - bcrypt: Modern password hashing for your software and your servers.
  - passlib: Comprehensive password hashing framework supporting over 30 schemes.
  - argon2-cffi: The secure Argon2 password hashing algorithm.
  - legacycrypt: `ctypes` wrapper to the POSIX crypt library call and associated functionality.
  - crypt_r: Fork of the `crypt` module, wrapper to the *`crypt_r(3)`* library call and associated functionality.
  - standard-crypt and deprecated-crypt-alternative: Use the redistribution of `crypt` and reimplementation of `_crypt` libraries from PyPI.

- `imghdr`: The filetype, puremagic, or python-magic libraries should be used as replacements. For example, the `puremagic.what()` function can be used to replace the `imghdr.what()` function for all file formats that were supported by `imghdr`.
  - standard-imghdr: Use the redistribution of `imghdr` library from PyPI.

- `mailcap`: Use the `mimetypes` module instead.
  - standard-mailcap: Use the redistribution of `mailcap` library from PyPI.

- `msilib`

- `nis`

- `nntplib`: Use the pynntp library from PyPI instead.
  - standard-nntplib: Use the redistribution of `nntplib` library from PyPI.

- `ossaudiodev`: For audio playback, use the pygame library from PyPI instead.

- `pipes`: Use the `subprocess` module instead. Use `shlex.quote()` to replace the undocumented `pipes.quote` function.
  - standard-pipes: Use the redistribution of `pipes` library from PyPI.

- `sndhdr`: The filetype, puremagic, or python-magic libraries should be used as replacements.
  - standard-sndhdr: Use the redistribution of `sndhdr` library from PyPI.

- `spwd`: Use the python-pam library from PyPI instead.
- `sunau`
  - standard-sunau: Use the redistribution of `sunau` library from PyPI.
- `telnetlib`, Use the telnetlib3 or Exscript libraries from PyPI instead.
  - standard-telnetlib: Use the redistribution of `telnetlib` library from PyPI.
- `uu`: Use the `base64` module instead, as a modern alternative.
  - standard-uu: Use the redistribution of `uu` library from PyPI.
- `xdrlib`
  - standard-xdrlib: Use the redistribution of `xdrlib` library from PyPI.

(Contributed by Victor Stinner and Zachary Ware in gh-104773 and gh-104780.)

7.2 2to3

- Remove the **`2to3`** program and the `lib2to3` module, previously deprecated in Python 3.11. (Contributed by Victor Stinner in gh-104780.)

7.3 builtins

- Remove support for chained `classmethod` descriptors (introduced in gh-63272). These can no longer be used to wrap other descriptors, such as `property`. The core design of this feature was flawed and led to several problems. To "pass-through" a `classmethod`, consider using the `__wrapped__` attribute that was added in Python 3.10. (Contributed by Raymond Hettinger in gh-89519.)
- Raise a `RuntimeError` when calling `frame.clear()` on a suspended frame (as has always been the case for an executing frame). (Contributed by Irit Katriel in gh-79932.)

7.4 configparser

- Remove the undocumented `LegacyInterpolation` class, deprecated in the docstring since Python 3.2, and at runtime since Python 3.11. (Contributed by Hugo van Kemenade in gh-104886.)

7.5 importlib.metadata

- Remove deprecated subscript (`__getitem__()`) access for EntryPoint objects. (Contributed by Jason R. Coombs in gh-113175.)

7.6 locale

- Remove the `locale.resetlocale()` function, deprecated in Python 3.11. Use `locale.setlocale(locale.LC_ALL, "")` instead. (Contributed by Victor Stinner in gh-104783.)

7.7 opcode

- Move `opcode.ENABLE_SPECIALIZATION` to `_opcode.ENABLE_SPECIALIZATION`. This field was added in 3.12, it was never documented, and is not intended for external use. (Contributed by Irit Katriel in gh-105481.)
- Remove `opcode.is_pseudo()`, `opcode.MIN_PSEUDO_OPCODE`, and `opcode.MAX_PSEUDO_OPCODE`, which were added in Python 3.12, but were neither documented nor exposed through `dis`, and were not intended to be used externally. (Contributed by Irit Katriel in gh-105481.)

7.8 optparse

- This module is no longer considered soft deprecated. While `argparse` remains preferred for new projects that aren't using a third party command line argument processing library, there are aspects of the way `argparse` works that mean the lower level `optparse` module may provide a better foundation for *writing* argument processing libraries, and for implementing command line applications which adhere more strictly than `argparse` does to various Unix command line processing conventions that originate in the behaviour of the C `getopt()` function . (Contributed by Alyssa Coghlan and Serhiy Storchaka in gh-126180.)

7.9 pathlib

- Remove the ability to use `Path` objects as context managers. This functionality was deprecated and has had no effect since Python 3.9. (Contributed by Barney Gale in gh-83863.)

7.10 re

- Remove the undocumented, deprecated, and broken `re.template()` function and `re.TEMPLATE` / `re.T` flag. (Contributed by Serhiy Storchaka and Nikita Sobolev in gh-105687.)

7.11 tkinter.tix

- Remove the `tkinter.tix` module, deprecated in Python 3.6. The third-party Tix library which the module wrapped is unmaintained. (Contributed by Zachary Ware in gh-75552.)

7.12 turtle

- Remove the `RawTurtle.settiltangle()` method, deprecated in the documentation since Python 3.1 and at runtime since Python 3.11. (Contributed by Hugo van Kemenade in gh-104876.)

7.13 typing

- Remove the `typing.io` and `typing.re` namespaces, deprecated since Python 3.8. The items in those namespaces can be imported directly from the `typing` module. (Contributed by Sebastian Rittau in gh-92871.)
- Remove the keyword-argument method of creating `TypedDict` types, deprecated in Python 3.11. (Contributed by Tomas Roun in gh-104786.)

7.14 unittest

- Remove the following `unittest` functions, deprecated in Python 3.11:
  - `unittest.findTestCases()`
  - `unittest.makeSuite()`
  - `unittest.getTestCaseNames()`

  Use `TestLoader` methods instead:
  - `loadTestsFromModule()`
  - `loadTestsFromTestCase()`
  - `getTestCaseNames()`

  (Contributed by Hugo van Kemenade in gh-104835.)
- Remove the untested and undocumented `TestProgram.usageExit()` method, deprecated in Python 3.11. (Contributed by Hugo van Kemenade in gh-104992.)

7.15 urllib

- Remove the *cafile*, *capath*, and *cadefault* parameters of the `urllib.request.urlopen()` function, deprecated in Python 3.6. Use the *context* parameter instead with an `SSLContext` instance. The `ssl.SSLContext.load_cert_chain()` function can be used to load specific certificates, or let `ssl.create_default_context()` select the operating system's trusted certificate authority (CA) certificates. (Contributed by Victor Stinner in gh-105382.)

7.16 webbrowser

- Remove the untested and undocumented `MacOSX` class, deprecated in Python 3.11. Use the `MacOSXOSAScript` class (introduced in Python 3.2) instead. (Contributed by Hugo van Kemenade in gh-104804.)
- Remove the deprecated `MacOSXOSAScript._name` attribute. Use the `MacOSXOSAScript.name` attribute instead. (Contributed by Nikita Sobolev in gh-105546.)

8 New Deprecations

- User-defined functions:
  - Deprecate assignment to a function's `__code__` attribute, where the new code object's type does not match the function's type. The different types are: plain function, generator, async generator, and coroutine. (Contributed by Irit Katriel in gh-81137.)
- `array`:
  - Deprecate the `'u'` format code (`wchar_t`) at runtime. This format code has been deprecated in documentation since Python 3.3, and will be removed in Python 3.16. Use the `'w'` format code (`Py_UCS4`) for Unicode characters instead. (Contributed by Hugo van Kemenade in gh-80480.)
- `ctypes`:
  - Deprecate the undocumented `SetPointerType()` function, to be removed in Python 3.15. (Contributed by Victor Stinner in gh-105733.)
  - Soft-deprecate the `ARRAY()` function in favour of `type * length` multiplication. (Contributed by Victor Stinner in gh-105733.)
- `decimal`:
  - Deprecate the non-standard and undocumented `Decimal` format specifier `'N'`, which is only supported in the `decimal` module's C implementation. (Contributed by Serhiy Storchaka in gh-89902.)
- `dis`:
  - Deprecate the `HAVE_ARGUMENT` separator. Check membership in `hasarg` instead. (Contributed by Irit Katriel in gh-109319.)
- `gettext`:
  - Deprecate non-integer numbers as arguments to functions and methods that consider plural forms in the `gettext` module, even if no translation was found. (Contributed by Serhiy Storchaka in gh-88434.)
- `glob`:
  - Deprecate the undocumented `glob0()` and `glob1()` functions. Use `glob()` and pass a path-like object specifying the root directory to the *root_dir* parameter instead. (Contributed by Barney Gale in gh-117337.)
- `http.server`:
  - Deprecate `CGIHTTPRequestHandler`, to be removed in Python 3.15. Process-based CGI HTTP servers have been out of favor for a very long time. This code was outdated, unmaintained, and rarely used. It has a high potential for both security and functionality bugs. (Contributed by Gregory P. Smith in gh-109096.)

- Deprecate the `--cgi` flag to the **`python -m http.server`** command-line interface, to be removed in Python 3.15. (Contributed by Gregory P. Smith in gh-109096.)

- `mimetypes`:
  - Soft-deprecate file path arguments to `guess_type()`, use `guess_file_type()` instead. (Contributed by Serhiy Storchaka in gh-66543.)

- `re`:
  - Deprecate passing the optional *maxsplit*, *count*, or *flags* arguments as positional arguments to the module-level `split()`, `sub()`, and `subn()` functions. These parameters will become keyword-only in a future version of Python. (Contributed by Serhiy Storchaka in gh-56166.)

- `pathlib`:
  - Deprecate `PurePath.is_reserved()`, to be removed in Python 3.15. Use `os.path.isreserved()` to detect reserved paths on Windows. (Contributed by Barney Gale in gh-88569.)

- `platform`:
  - Deprecate `java_ver()`, to be removed in Python 3.15. This function is only useful for Jython support, has a confusing API, and is largely untested. (Contributed by Nikita Sobolev in gh-116349.)

- `pydoc`:
  - Deprecate the undocumented `ispackage()` function. (Contributed by Zackery Spytz in gh-64020.)

- `sqlite3`:
  - Deprecate passing more than one positional argument to the `connect()` function and the `Connection` constructor. The remaining parameters will become keyword-only in Python 3.15. (Contributed by Erlend E. Aasland in gh-107948.)
  - Deprecate passing name, number of arguments, and the callable as keyword arguments for `Connection.create_function()` and `Connection.create_aggregate()` These parameters will become positional-only in Python 3.15. (Contributed by Erlend E. Aasland in gh-108278.)
  - Deprecate passing the callback callable by keyword for the `set_authorizer()`, `set_progress_handler()`, and `set_trace_callback()` `Connection` methods. The callback callables will become positional-only in Python 3.15. (Contributed by Erlend E. Aasland in gh-108278.)

- `sys`:
  - Deprecate the `_enablelegacywindowsfsencoding()` function, to be removed in Python 3.16. Use the `PYTHONLEGACYWINDOWSFSENCODING` environment variable instead. (Contributed by Inada Naoki in gh-73427.)

- `tarfile`:
  - Deprecate the undocumented and unused `TarFile.tarfile` attribute, to be removed in Python 3.16. (Contributed in gh-115256.)

- `traceback`:
  - Deprecate the `TracebackException.exc_type` attribute. Use `TracebackException.exc_type_str` instead. (Contributed by Irit Katriel in gh-112332.)

- `typing`:
  - Deprecate the undocumented keyword argument syntax for creating `NamedTuple` classes (e.g. `Point = NamedTuple("Point", x=int, y=int)`), to be removed in Python 3.15. Use the class-based syntax or the functional syntax instead. (Contributed by Alex Waygood in gh-105566.)
  - Deprecate omitting the *fields* parameter when creating a `NamedTuple` or `typing.TypedDict` class, and deprecate passing `None` to the *fields* parameter of both types. Python 3.15 will require a valid sequence for the *fields* parameter. To create a NamedTuple class with zero fields, use `class NT(NamedTuple): pass` or `NT = NamedTuple("NT", ())`. To create a TypedDict class with zero

fields, use `class TD(TypedDict): pass` or `TD = TypedDict("TD", {})`. (Contributed by Alex Waygood in gh-105566 and gh-105570.)

- Deprecate the `typing.no_type_check_decorator()` decorator function, to be removed in Python 3.15. After eight years in the `typing` module, it has yet to be supported by any major type checker. (Contributed by Alex Waygood in gh-106309.)
- Deprecate `typing.AnyStr`. In Python 3.16, it will be removed from `typing.__all__`, and a `DeprecationWarning` will be emitted at runtime when it is imported or accessed. It will be removed entirely in Python 3.18. Use the new type parameter syntax instead. (Contributed by Michael The in gh-107116.)

- `wave`:
  - Deprecate the `getmark()`, `setmark()`, and `getmarkers()` methods of the `Wave_read` and `Wave_write` classes, to be removed in Python 3.15. (Contributed by Victor Stinner in gh-105096.)

8.1 Pending Removal in Python 3.14

- `argparse`: The *type*, *choices*, and *metavar* parameters of `argparse.BooleanOptionalAction` are deprecated and will be removed in 3.14. (Contributed by Nikita Sobolev in gh-92248.)
- `ast`: The following features have been deprecated in documentation since Python 3.8, now cause a `DeprecationWarning` to be emitted at runtime when they are accessed or used, and will be removed in Python 3.14:
  - `ast.Num`
  - `ast.Str`
  - `ast.Bytes`
  - `ast.NameConstant`
  - `ast.Ellipsis`

  Use `ast.Constant` instead. (Contributed by Serhiy Storchaka in gh-90953.)
- `asyncio`:
  - The child watcher classes `MultiLoopChildWatcher`, `FastChildWatcher`, `AbstractChildWatcher` and `SafeChildWatcher` are deprecated and will be removed in Python 3.14. (Contributed by Kumar Aditya in gh-94597.)
  - `asyncio.set_child_watcher()`, `asyncio.get_child_watcher()`, `asyncio.AbstractEventLoopPolicy.set_child_watcher()` and `asyncio.AbstractEventLoopPolicy.get_child_watcher()` are deprecated and will be removed in Python 3.14. (Contributed by Kumar Aditya in gh-94597.)
  - The `get_event_loop()` method of the default event loop policy now emits a `DeprecationWarning` if there is no current event loop set and it decides to create one. (Contributed by Serhiy Storchaka and Guido van Rossum in gh-100160.)
- `email`: Deprecated the *isdst* parameter in `email.utils.localtime()`. (Contributed by Alan Williams in gh-72346.)
- `importlib.abc` deprecated classes:
  - `importlib.abc.ResourceReader`
  - `importlib.abc.Traversable`
  - `importlib.abc.TraversableResources`

  Use `importlib.resources.abc` classes instead:
  - `importlib.resources.abc.Traversable`
  - `importlib.resources.abc.TraversableResources`

(Contributed by Jason R. Coombs and Hugo van Kemenade in gh-93963.)

- `itertools` had undocumented, inefficient, historically buggy, and inconsistent support for copy, deepcopy, and pickle operations. This will be removed in 3.14 for a significant reduction in code volume and maintenance burden. (Contributed by Raymond Hettinger in gh-101588.)
- `multiprocessing`: The default start method will change to a safer one on Linux, BSDs, and other non-macOS POSIX platforms where `'fork'` is currently the default (gh-84559). Adding a runtime warning about this was deemed too disruptive as the majority of code is not expected to care. Use the `get_context()` or `set_start_method()` APIs to explicitly specify when your code *requires* `'fork'`. See multiprocessing-start-methods.
- `pathlib`: `is_relative_to()` and `relative_to()`: passing additional arguments is deprecated.
- `pkgutil`: `find_loader()` and `get_loader()` now raise `DeprecationWarning`; use `importlib.util.find_spec()` instead. (Contributed by Nikita Sobolev in gh-97850.)
- `pty`:
  - `master_open()`: use `pty.openpty()`.
  - `slave_open()`: use `pty.openpty()`.
- `sqlite3`:
  - `version` and `version_info`.
  - `execute()` and `executemany()` if named placeholders are used and *parameters* is a sequence instead of a `dict`.
- `urllib`: `urllib.parse.Quoter` is deprecated: it was not intended to be a public API. (Contributed by Gregory P. Smith in gh-88168.)

8.2 Pending Removal in Python 3.15

- The import system:
  - Setting `__cached__` on a module while failing to set `__spec__.cached` is deprecated. In Python 3.15, `__cached__` will cease to be set or take into consideration by the import system or standard library. (gh-97879)
  - Setting `__package__` on a module while failing to set `__spec__.parent` is deprecated. In Python 3.15, `__package__` will cease to be set or take into consideration by the import system or standard library. (gh-97879)
- `ctypes`:
  - The undocumented `ctypes.SetPointerType()` function has been deprecated since Python 3.13.
- `http.server`:
  - The obsolete and rarely used `CGIHTTPRequestHandler` has been deprecated since Python 3.13. No direct replacement exists. *Anything* is better than CGI to interface a web server with a request handler.
  - The `--cgi` flag to the **`python -m http.server`** command-line interface has been deprecated since Python 3.13.
- `importlib`:
  - `load_module()` method: use `exec_module()` instead.
- `locale`:
  - The `getdefaultlocale()` function has been deprecated since Python 3.11. Its removal was originally planned for Python 3.13 (gh-90817), but has been postponed to Python 3.15. Use `getlocale()`, `setlocale()`, and `getencoding()` instead. (Contributed by Hugo van Kemenade in gh-111187.)
- `pathlib`:

  - `PurePath.is_reserved()` has been deprecated since Python 3.13. Use `os.path.isreserved()` to detect reserved paths on Windows.

- `platform`:
  - `java_ver()` has been deprecated since Python 3.13. This function is only useful for Jython support, has a confusing API, and is largely untested.

- `sysconfig`:
  - The *check_home* argument of `sysconfig.is_python_build()` has been deprecated since Python 3.12.

- `threading`:
  - `RLock()` will take no arguments in Python 3.15. Passing any arguments has been deprecated since Python 3.14, as the Python version does not permit any arguments, but the C version allows any number of positional or keyword arguments, ignoring every argument.

- `types`:
  - `types.CodeType`: Accessing `co_lnotab` was deprecated in **PEP 626** since 3.10 and was planned to be removed in 3.12, but it only got a proper `DeprecationWarning` in 3.12. May be removed in 3.15. (Contributed by Nikita Sobolev in gh-101866.)

- `typing`:
  - The undocumented keyword argument syntax for creating `NamedTuple` classes (e.g. `Point = NamedTuple("Point", x=int, y=int)`) has been deprecated since Python 3.13. Use the class-based syntax or the functional syntax instead.
  - When using the functional syntax of `TypedDict`s, failing to pass a value to the *fields* parameter (`TD = TypedDict("TD")`) or passing `None` (`TD = TypedDict("TD", None)`) has been deprecated since Python 3.13. Use `class TD(TypedDict): pass` or `TD = TypedDict("TD", {})` to create a TypedDict with zero field.
  - The `typing.no_type_check_decorator()` decorator function has been deprecated since Python 3.13. After eight years in the `typing` module, it has yet to be supported by any major type checker.

- `wave`:
  - The `getmark()`, `setmark()`, and `getmarkers()` methods of the `Wave_read` and `Wave_write` classes have been deprecated since Python 3.13.

8.3 Pending removal in Python 3.16

- The import system:
  - Setting `__loader__` on a module while failing to set `__spec__.loader` is deprecated. In Python 3.16, `__loader__` will cease to be set or taken into consideration by the import system or the standard library.

- `array`:
  - The `'u'` format code (`wchar_t`) has been deprecated in documentation since Python 3.3 and at runtime since Python 3.13. Use the `'w'` format code (`Py_UCS4`) for Unicode characters instead.

- `asyncio`:
  - `asyncio.iscoroutinefunction()` is deprecated and will be removed in Python 3.16, use `inspect.iscoroutinefunction()` instead. (Contributed by Jiahao Li and Kumar Aditya in gh-122875.)

- `builtins`:
  - Bitwise inversion on boolean types, `~True` or `~False` has been deprecated since Python 3.12, as it produces surprising and unintuitive results (`-2` and `-1`). Use `not x` instead for the logical negation of a Boolean. In the rare case that you need the bitwise inversion of the underlying integer, convert to `int` explicitly (`~int(x)`).

- shutil:
  - The ExecError exception has been deprecated since Python 3.14. It has not been used by any function in shutil since Python 3.4, and is now an alias of RuntimeError.
- symtable:
  - The Class.get_methods method has been deprecated since Python 3.14.
- sys:
  - The _enablelegacywindowsfsencoding() function has been deprecated since Python 3.13. Use the PYTHONLEGACYWINDOWSFSENCODING environment variable instead.
- tarfile:
  - The undocumented and unused TarFile.tarfile attribute has been deprecated since Python 3.13.

8.4 Pending removal in Python 3.17

- collections.abc:
  - collections.abc.ByteString is scheduled for removal in Python 3.17.

    Use isinstance(obj, collections.abc.Buffer) to test if obj implements the buffer protocol at runtime. For use in type annotations, either use Buffer or a union that explicitly specifies the types your code supports (e.g., bytes | bytearray | memoryview).

    ByteString was originally intended to be an abstract class that would serve as a supertype of both bytes and bytearray. However, since the ABC never had any methods, knowing that an object was an instance of ByteString never actually told you anything useful about the object. Other common buffer types such as memoryview were also never understood as subtypes of ByteString (either at runtime or by static type checkers).

    See **PEP 688** for more details. (Contributed by Shantanu Jain in gh-91896.)
- typing:
  - Before Python 3.14, old-style unions were implemented using the private class typing._UnionGenericAlias. This class is no longer needed for the implementation, but it has been retained for backward compatibility, with removal scheduled for Python 3.17. Users should use documented introspection helpers like typing.get_origin() and typing.get_args() instead of relying on private implementation details.
  - typing.ByteString, deprecated since Python 3.9, is scheduled for removal in Python 3.17.

    Use isinstance(obj, collections.abc.Buffer) to test if obj implements the buffer protocol at runtime. For use in type annotations, either use Buffer or a union that explicitly specifies the types your code supports (e.g., bytes | bytearray | memoryview).

    ByteString was originally intended to be an abstract class that would serve as a supertype of both bytes and bytearray. However, since the ABC never had any methods, knowing that an object was an instance of ByteString never actually told you anything useful about the object. Other common buffer types such as memoryview were also never understood as subtypes of ByteString (either at runtime or by static type checkers).

    See **PEP 688** for more details. (Contributed by Shantanu Jain in gh-91896.)

8.5 Pending Removal in Future Versions

The following APIs will be removed in the future, although there is currently no date scheduled for their removal.

- argparse: Nesting argument groups and nesting mutually exclusive groups are deprecated.
- builtins:
  - bool(NotImplemented).

  - Generators: `throw(type, exc, tb)` and `athrow(type, exc, tb)` signature is deprecated: use `throw(exc)` and `athrow(exc)` instead, the single argument signature.
  - Currently Python accepts numeric literals immediately followed by keywords, for example `0in x`, `1or x`, `0if 1else 2`. It allows confusing and ambiguous expressions like `[0x1for x in y]` (which can be interpreted as `[0x1 for x in y]` or `[0x1f or x in y]`). A syntax warning is raised if the numeric literal is immediately followed by one of keywords `and`, `else`, `for`, `if`, `in`, `is` and `or`. In a future release it will be changed to a syntax error. (gh-87999)
  - Support for `__index__()` and `__int__()` method returning non-int type: these methods will be required to return an instance of a strict subclass of `int`.
  - Support for `__float__()` method returning a strict subclass of `float`: these methods will be required to return an instance of `float`.
  - Support for `__complex__()` method returning a strict subclass of `complex`: these methods will be required to return an instance of `complex`.
  - Delegation of `int()` to `__trunc__()` method.
  - Passing a complex number as the *real* or *imag* argument in the `complex()` constructor is now deprecated; it should only be passed as a single positional argument. (Contributed by Serhiy Storchaka in gh-109218.)
- `calendar`: `calendar.January` and `calendar.February` constants are deprecated and replaced by `calendar.JANUARY` and `calendar.FEBRUARY`. (Contributed by Prince Roshan in gh-103636.)
- `codeobject.co_lnotab`: use the `codeobject.co_lines()` method instead.
- `datetime`:
  - `utcnow()`: use `datetime.datetime.now(tz=datetime.UTC)`.
  - `utcfromtimestamp()`: use `datetime.datetime.fromtimestamp(timestamp, tz=datetime.UTC)`.
- `gettext`: Plural value must be an integer.
- `importlib`:
  - `cache_from_source()` *debug_override* parameter is deprecated: use the *optimization* parameter instead.
- `importlib.metadata`:
  - `EntryPoints` tuple interface.
  - Implicit `None` on return values.
- `logging`: the `warn()` method has been deprecated since Python 3.3, use `warning()` instead.
- `mailbox`: Use of StringIO input and text mode is deprecated, use BytesIO and binary mode instead.
- `os`: Calling `os.register_at_fork()` in multi-threaded process.
- `pydoc.ErrorDuringImport`: A tuple value for *exc_info* parameter is deprecated, use an exception instance.
- `re`: More strict rules are now applied for numerical group references and group names in regular expressions. Only sequence of ASCII digits is now accepted as a numerical reference. The group name in bytes patterns and replacement strings can now only contain ASCII letters and digits and underscore. (Contributed by Serhiy Storchaka in gh-91760.)
- `sre_compile`, `sre_constants` and `sre_parse` modules.
- `shutil`: `rmtree()`'s *onerror* parameter is deprecated in Python 3.12; use the *onexc* parameter instead.
- `ssl` options and protocols:
  - `ssl.SSLContext` without protocol argument is deprecated.
  - `ssl.SSLContext`: `set_npn_protocols()` and `selected_npn_protocol()` are deprecated: use ALPN instead.

  - `ssl.OP_NO_SSL*` options
  - `ssl.OP_NO_TLS*` options
  - `ssl.PROTOCOL_SSLv3`
  - `ssl.PROTOCOL_TLS`
  - `ssl.PROTOCOL_TLSv1`
  - `ssl.PROTOCOL_TLSv1_1`
  - `ssl.PROTOCOL_TLSv1_2`
  - `ssl.TLSVersion.SSLv3`
  - `ssl.TLSVersion.TLSv1`
  - `ssl.TLSVersion.TLSv1_1`
- `threading` methods:
  - `threading.Condition.notifyAll()`: use `notify_all()`.
  - `threading.Event.isSet()`: use `is_set()`.
  - `threading.Thread.isDaemon()`, `threading.Thread.setDaemon()`: use `threading.Thread.daemon` attribute.
  - `threading.Thread.getName()`, `threading.Thread.setName()`: use `threading.Thread.name` attribute.
  - `threading.currentThread()`: use `threading.current_thread()`.
  - `threading.activeCount()`: use `threading.active_count()`.
- `typing.Text` (gh-92332).
- `unittest.IsolatedAsyncioTestCase`: it is deprecated to return a value that is not `None` from a test case.
- `urllib.parse` deprecated functions: `urlparse()` instead
  - `splitattr()`
  - `splithost()`
  - `splitnport()`
  - `splitpasswd()`
  - `splitport()`
  - `splitquery()`
  - `splittag()`
  - `splittype()`
  - `splituser()`
  - `splitvalue()`
  - `to_bytes()`
- `urllib.request`: `URLopener` and `FancyURLopener` style of invoking requests is deprecated. Use newer `urlopen()` functions and methods.
- `wsgiref`: `SimpleHandler.stdout.write()` should not do partial writes.
- `xml.etree.ElementTree`: Testing the truth value of an `Element` is deprecated. In a future release it will always return `True`. Prefer explicit `len(elem)` or `elem is not None` tests instead.
- `zipimport.zipimporter.load_module()` is deprecated: use `exec_module()` instead.

9 CPython Bytecode Changes

- The oparg of `YIELD_VALUE` is now `1` if the yield is part of a yield-from or await, and `0` otherwise. The oparg of `RESUME` was changed to add a bit indicating if the except-depth is 1, which is needed to optimize closing of generators. (Contributed by Irit Katriel in gh-111354.)

10 C API Changes

10.1 New Features

- Add the PyMonitoring C API for generating **PEP 669** monitoring events:
  - `PyMonitoringState`
  - `PyMonitoring_FirePyStartEvent()`
  - `PyMonitoring_FirePyResumeEvent()`
  - `PyMonitoring_FirePyReturnEvent()`
  - `PyMonitoring_FirePyYieldEvent()`
  - `PyMonitoring_FireCallEvent()`
  - `PyMonitoring_FireLineEvent()`
  - `PyMonitoring_FireJumpEvent()`
  - `PyMonitoring_FireBranchEvent()`
  - `PyMonitoring_FireCReturnEvent()`
  - `PyMonitoring_FirePyThrowEvent()`
  - `PyMonitoring_FireRaiseEvent()`
  - `PyMonitoring_FireCRaiseEvent()`
  - `PyMonitoring_FireReraiseEvent()`
  - `PyMonitoring_FireExceptionHandledEvent()`
  - `PyMonitoring_FirePyUnwindEvent()`
  - `PyMonitoring_FireStopIterationEvent()`
  - `PyMonitoring_EnterScope()`
  - `PyMonitoring_ExitScope()`

  (Contributed by Irit Katriel in gh-111997).
- Add `PyMutex`, a lightweight mutex that occupies a single byte, and the new `PyMutex_Lock()` and `PyMutex_Unlock()` functions. `PyMutex_Lock()` will release the GIL (if currently held) if the operation needs to block. (Contributed by Sam Gross in gh-108724.)
- Add the PyTime C API to provide access to system clocks:
  - `PyTime_t`.
  - `PyTime_MIN` and `PyTime_MAX`.
  - `PyTime_AsSecondsDouble()`.
  - `PyTime_Monotonic()`.
  - `PyTime_MonotonicRaw()`.
  - `PyTime_PerfCounter()`.
  - `PyTime_PerfCounterRaw()`.

  - `PyTime_Time()`.
  - `PyTime_TimeRaw()`.

  (Contributed by Victor Stinner and Petr Viktorin in gh-110850.)

- Add the `PyDict_ContainsString()` function with the same behavior as `PyDict_Contains()`, but *key* is specified as a `const char*` UTF-8 encoded bytes string, rather than a `PyObject*`. (Contributed by Victor Stinner in gh-108314.)
- Add the `PyDict_GetItemRef()` and `PyDict_GetItemStringRef()` functions, which behave similarly to `PyDict_GetItemWithError()`, but return a strong reference instead of a borrowed reference. Moreover, these functions return `-1` on error, removing the need to check `PyErr_Occurred()`. (Contributed by Victor Stinner in gh-106004.)
- Add the `PyDict_SetDefaultRef()` function, which behaves similarly to `PyDict_SetDefault()`, but returns a strong reference instead of a borrowed reference. This function returns `-1` on error, `0` on insertion, and `1` if the key was already present in the dictionary. (Contributed by Sam Gross in gh-112066.)
- Add the `PyDict_Pop()` and `PyDict_PopString()` functions to remove a key from a dictionary and optionally return the removed value. This is similar to `dict.pop()`, though there is no default value, and `KeyError` is not raised for missing keys. (Contributed by Stefan Behnel and Victor Stinner in gh-111262.)
- Add the `PyMapping_GetOptionalItem()` and `PyMapping_GetOptionalItemString()` functions as alternatives to `PyObject_GetItem()` and `PyMapping_GetItemString()` respectively. The new functions do not raise `KeyError` if the requested key is missing from the mapping. These variants are more convenient and faster if a missing key should not be treated as a failure. (Contributed by Serhiy Storchaka in gh-106307.)
- Add the `PyObject_GetOptionalAttr()` and `PyObject_GetOptionalAttrString()` functions as alternatives to `PyObject_GetAttr()` and `PyObject_GetAttrString()` respectively. The new functions do not raise `AttributeError` if the requested attribute is not found on the object. These variants are more convenient and faster if the missing attribute should not be treated as a failure. (Contributed by Serhiy Storchaka in gh-106521.)
- Add the `PyErr_FormatUnraisable()` function as an extension to `PyErr_WriteUnraisable()` that allows customizing the warning message. (Contributed by Serhiy Storchaka in gh-108082.)
- Add new functions that return a strong reference instead of a borrowed reference for frame locals, globals, and builtins, as part of *PEP 667*:
  - `PyEval_GetFrameBuiltins()` replaces `PyEval_GetBuiltins()`
  - `PyEval_GetFrameGlobals()` replaces `PyEval_GetGlobals()`
  - `PyEval_GetFrameLocals()` replaces `PyEval_GetLocals()`

  (Contributed by Mark Shannon and Tian Gao in gh-74929.)
- Add the `Py_GetConstant()` and `Py_GetConstantBorrowed()` functions to get strong or borrowed references to constants. For example, `Py_GetConstant(Py_CONSTANT_ZERO)` returns a strong reference to the constant zero. (Contributed by Victor Stinner in gh-115754.)
- Add the `PyImport_AddModuleRef()` function as a replacement for `PyImport_AddModule()` that returns a strong reference instead of a borrowed reference. (Contributed by Victor Stinner in gh-105922.)
- Add the `Py_IsFinalizing()` function to check whether the main Python interpreter is shutting down. (Contributed by Victor Stinner in gh-108014.)
- Add the `PyList_GetItemRef()` function as a replacement for `PyList_GetItem()` that returns a strong reference instead of a borrowed reference. (Contributed by Sam Gross in gh-114329.)
- Add the `PyList_Extend()` and `PyList_Clear()` functions, mirroring the Python `list.extend()` and `list.clear()` methods. (Contributed by Victor Stinner in gh-111138.)
- Add the `PyLong_AsInt()` function. It behaves similarly to `PyLong_AsLong()`, but stores the result in a C `int` instead of a C `long`. (Contributed by Victor Stinner in gh-108014.)

- Add the `PyLong_AsNativeBytes()`, `PyLong_FromNativeBytes()`, and `PyLong_FromUnsignedNativeBytes()` functions to simplify converting between native integer types and Python `int` objects. (Contributed by Steve Dower in gh-111140.)
- Add `PyModule_Add()` function, which is similar to `PyModule_AddObjectRef()` and `PyModule_AddObject()`, but always steals a reference to the value. (Contributed by Serhiy Storchaka in gh-86493.)
- Add the `PyObject_GenericHash()` function that implements the default hashing function of a Python object. (Contributed by Serhiy Storchaka in gh-113024.)
- Add the `Py_HashPointer()` function to hash a raw pointer. (Contributed by Victor Stinner in gh-111545.)
- Add the `PyObject_VisitManagedDict()` and `PyObject_ClearManagedDict()` functions. which must be called by the traverse and clear functions of a type using the `Py_TPFLAGS_MANAGED_DICT` flag. The pythoncapi-compat project can be used to use these functions with Python 3.11 and 3.12. (Contributed by Victor Stinner in gh-107073.)
- Add the `PyRefTracer_SetTracer()` and `PyRefTracer_GetTracer()` functions, which enable tracking object creation and destruction in the same way that the `tracemalloc` module does. (Contributed by Pablo Galindo in gh-93502.)
- Add the `PySys_AuditTuple()` function as an alternative to `PySys_Audit()` that takes event arguments as a Python `tuple` object. (Contributed by Victor Stinner in gh-85283.)
- Add the `PyThreadState_GetUnchecked()` function as an alternative to `PyThreadState_Get()` that doesn't kill the process with a fatal error if it is `NULL`. The caller is responsible for checking if the result is `NULL`. (Contributed by Victor Stinner in gh-108867.)
- Add the `PyType_GetFullyQualifiedName()` function to get the type's fully qualified name. The module name is prepended if `type.__module__` is a string and is not equal to either `'builtins'` or `'__main__'`. (Contributed by Victor Stinner in gh-111696.)
- Add the `PyType_GetModuleName()` function to get the type's module name. This is equivalent to getting the `type.__module__` attribute. (Contributed by Eric Snow and Victor Stinner in gh-111696.)
- Add the `PyUnicode_EqualToUTF8AndSize()` and `PyUnicode_EqualToUTF8()` functions to compare a Unicode object with a `const char*` UTF-8 encoded string and `1` if they are equal or `0` otherwise. These functions do not raise exceptions. (Contributed by Serhiy Storchaka in gh-110289.)
- Add the `PyWeakref_GetRef()` function as an alternative to `PyWeakref_GetObject()` that returns a strong reference or `NULL` if the referent is no longer live. (Contributed by Victor Stinner in gh-105927.)
- Add fixed variants of functions which silently ignore errors:
  - `PyObject_HasAttrWithError()` replaces `PyObject_HasAttr()`.
  - `PyObject_HasAttrStringWithError()` replaces `PyObject_HasAttrString()`.
  - `PyMapping_HasKeyWithError()` replaces `PyMapping_HasKey()`.
  - `PyMapping_HasKeyStringWithError()` replaces `PyMapping_HasKeyString()`.

  The new functions return `-1` for errors and the standard `1` for true and `0` for false.

  (Contributed by Serhiy Storchaka in gh-108511.)

10.2 Changed C APIs

- The *keywords* parameter of `PyArg_ParseTupleAndKeywords()` and `PyArg_VaParseTupleAndKeywords()` now has type `char *const*` in C and `const char *const*` in C++, instead of `char**`. In C++, this makes these functions compatible with arguments of type `const char *const*`, `const char**`, or `char *const*` without an explicit type cast. In C, the functions only support arguments of type `char *const*`. This can be overridden with the `PY_CXX_CONST` macro. (Contributed by Serhiy Storchaka in gh-65210.)
- `PyArg_ParseTupleAndKeywords()` now supports non-ASCII keyword parameter names. (Contributed by Serhiy Storchaka in gh-110815.)

- The `PyCode_GetFirstFree()` function is now unstable API and is now named `PyUnstable_Code_GetFirstFree()`. (Contributed by Bogdan Romanyuk in gh-115781.)
- The `PyDict_GetItem()`, `PyDict_GetItemString()`, `PyMapping_HasKey()`, `PyMapping_HasKeyString()`, `PyObject_HasAttr()`, `PyObject_HasAttrString()`, and `PySys_GetObject()` functions, each of which clears all errors which occurred when calling them now reports these errors using `sys.unraisablehook()`. You may replace them with other functions as recommended in the documentation. (Contributed by Serhiy Storchaka in gh-106672.)
- Add support for the `%T`, `%#T`, `%N` and `%#N` formats to `PyUnicode_FromFormat()`:
  - `%T`: Get the fully qualified name of an object type
  - `%#T`: As above, but use a colon as the separator
  - `%N`: Get the fully qualified name of a type
  - `%#N`: As above, but use a colon as the separator

  See **PEP 737** for more information. (Contributed by Victor Stinner in gh-111696.)
- You no longer have to define the `PY_SSIZE_T_CLEAN` macro before including `Python.h` when using `#` formats in format codes. APIs accepting the format codes always use `Py_ssize_t` for `#` formats. (Contributed by Inada Naoki in gh-104922.)
- If Python is built in debug mode or `with assertions`, `PyTuple_SET_ITEM()` and `PyList_SET_ITEM()` now check the index argument with an assertion. (Contributed by Victor Stinner in gh-106168.)

10.3 Limited C API Changes

- The following functions are now included in the Limited C API:
  - `PyMem_RawMalloc()`
  - `PyMem_RawCalloc()`
  - `PyMem_RawRealloc()`
  - `PyMem_RawFree()`
  - `PySys_Audit()`
  - `PySys_AuditTuple()`
  - `PyType_GetModuleByDef()`

  (Contributed by Victor Stinner in gh-85283, gh-85283, and gh-116936.)
- Python built with `--with-trace-refs` (tracing references) now supports the Limited API. (Contributed by Victor Stinner in gh-108634.)

10.4 Removed C APIs

- Remove several functions, macros, variables, etc with names prefixed by `_Py` or `_PY` (which are considered private). If your project is affected by one of these removals and you believe that the removed API should remain available, please open a new issue to request a public C API and add `cc: @vstinner` to the issue to notify Victor Stinner. (Contributed by Victor Stinner in gh-106320.)
- Remove old buffer protocols deprecated in Python 3.0. Use bufferobjects instead.
  - `PyObject_CheckReadBuffer()`: Use `PyObject_CheckBuffer()` to test whether the object supports the buffer protocol. Note that `PyObject_CheckBuffer()` doesn't guarantee that `PyObject_GetBuffer()` will succeed. To test if the object is actually readable, see the next example of `PyObject_GetBuffer()`.
  - `PyObject_AsCharBuffer()`, `PyObject_AsReadBuffer()`: Use `PyObject_GetBuffer()` and `PyBuffer_Release()` instead:

```
Py_buffer view;
if (PyObject_GetBuffer(obj, &view, PyBUF_SIMPLE) < 0) {
    return NULL;
}
// Use `view.buf` and `view.len` to read from the buffer.
// You may need to cast buf as `(const char*)view.buf`.
PyBuffer_Release(&view);
```

- `PyObject_AsWriteBuffer()`: Use `PyObject_GetBuffer()` and `PyBuffer_Release()` instead:

```
Py_buffer view;
if (PyObject_GetBuffer(obj, &view, PyBUF_WRITABLE) < 0) {
    return NULL;
}
// Use `view.buf` and `view.len` to write to the buffer.
PyBuffer_Release(&view);
```

(Contributed by Inada Naoki in gh-85275.)

- Remove various functions deprecated in Python 3.9:
  - `PyEval_CallObject()`, `PyEval_CallObjectWithKeywords()`: Use `PyObject_CallNoArgs()` or `PyObject_Call()` instead.

> **Warning**
>
> In `PyObject_Call()`, positional arguments must be a `tuple` and must not be `NULL`, and keyword arguments must be a `dict` or `NULL`, whereas the removed functions checked argument types and accepted `NULL` positional and keyword arguments. To replace `PyEval_CallObjectWithKeywords(func, NULL, kwargs)` with `PyObject_Call()`, pass an empty tuple as positional arguments using `PyTuple_New(0)`.

  - `PyEval_CallFunction()`: Use `PyObject_CallFunction()` instead.
  - `PyEval_CallMethod()`: Use `PyObject_CallMethod()` instead.
  - `PyCFunction_Call()`: Use `PyObject_Call()` instead.

(Contributed by Victor Stinner in gh-105107.)

- Remove the following old functions to configure the Python initialization, deprecated in Python 3.11:
  - `PySys_AddWarnOptionUnicode()`: Use `PyConfig.warnoptions` instead.
  - `PySys_AddWarnOption()`: Use `PyConfig.warnoptions` instead.
  - `PySys_AddXOption()`: Use `PyConfig.xoptions` instead.
  - `PySys_HasWarnOptions()`: Use `PyConfig.xoptions` instead.
  - `PySys_SetPath()`: Set `PyConfig.module_search_paths` instead.
  - `Py_SetPath()`: Set `PyConfig.module_search_paths` instead.
  - `Py_SetStandardStreamEncoding()`: Set `PyConfig.stdio_encoding` instead, and set also maybe `PyConfig.legacy_windows_stdio` (on Windows).
  - `_Py_SetProgramFullPath()`: Set `PyConfig.executable` instead.

  Use the new `PyConfig` API of the Python Initialization Configuration instead (**PEP 587**), added to Python 3.8. (Contributed by Victor Stinner in gh-105145.)

- Remove `PyEval_AcquireLock()` and `PyEval_ReleaseLock()` functions, deprecated in Python 3.2. They didn't update the current thread state. They can be replaced with:

  - `PyEval_SaveThread()` and `PyEval_RestoreThread()`;
  - low-level `PyEval_AcquireThread()` and `PyEval_RestoreThread()`;
  - or `PyGILState_Ensure()` and `PyGILState_Release()`.

  (Contributed by Victor Stinner in gh-105182.)

- Remove the `PyEval_ThreadsInitialized()` function, deprecated in Python 3.9. Since Python 3.7, `Py_Initialize()` always creates the GIL: calling `PyEval_InitThreads()` does nothing and `PyEval_ThreadsInitialized()` always returns non-zero. (Contributed by Victor Stinner in gh-105182.)
- Remove the `_PyInterpreterState_Get()` alias to `PyInterpreterState_Get()` which was kept for backward compatibility with Python 3.8. The pythoncapi-compat project can be used to get `PyInterpreterState_Get()` on Python 3.8 and older. (Contributed by Victor Stinner in gh-106320.)
- Remove the private `_PyObject_FastCall()` function: use `PyObject_Vectorcall()` which is available since Python 3.8 (**PEP 590**). (Contributed by Victor Stinner in gh-106023.)
- Remove the `cpython/pytime.h` header file, which only contained private functions. (Contributed by Victor Stinner in gh-106316.)
- Remove the undocumented `PY_TIMEOUT_MAX` constant from the limited C API. (Contributed by Victor Stinner in gh-110014.)
- Remove the old trashcan macros `Py_TRASHCAN_SAFE_BEGIN` and `Py_TRASHCAN_SAFE_END`. Replace both with the new macros `Py_TRASHCAN_BEGIN` and `Py_TRASHCAN_END`. (Contributed by Irit Katriel in gh-105111.)

10.5 Deprecated C APIs

- Deprecate old Python initialization functions:
  - `PySys_ResetWarnOptions()`: Clear `sys.warnoptions` and `warnings.filters` instead.
  - `Py_GetExecPrefix()`: Get `sys.exec_prefix` instead.
  - `Py_GetPath()`: Get `sys.path` instead.
  - `Py_GetPrefix()`: Get `sys.prefix` instead.
  - `Py_GetProgramFullPath()`: Get `sys.executable` instead.
  - `Py_GetProgramName()`: Get `sys.executable` instead.
  - `Py_GetPythonHome()`: Get `PyConfig.home` or the `PYTHONHOME` environment variable instead.

  (Contributed by Victor Stinner in gh-105145.)
- Soft deprecate the `PyEval_GetBuiltins()`, `PyEval_GetGlobals()`, and `PyEval_GetLocals()` functions, which return a borrowed reference. (Soft deprecated as part of **PEP 667**.)
- Deprecate the `PyImport_ImportModuleNoBlock()` function, which is just an alias to `PyImport_ImportModule()` since Python 3.3. (Contributed by Victor Stinner in gh-105396.)
- Soft deprecate the `PyModule_AddObject()` function. It should be replaced with `PyModule_Add()` or `PyModule_AddObjectRef()`. (Contributed by Serhiy Storchaka in gh-86493.)
- Deprecate the old `Py_UNICODE` and `PY_UNICODE_TYPE` types and the `Py_UNICODE_WIDE` define. Use the `wchar_t` type directly instead. Since Python 3.3, `Py_UNICODE` and `PY_UNICODE_TYPE` are just aliases to `wchar_t`. (Contributed by Victor Stinner in gh-105156.)
- Deprecate the `PyWeakref_GetObject()` and `PyWeakref_GET_OBJECT()` functions, which return a borrowed reference. Replace them with the new `PyWeakref_GetRef()` function, which returns a strong reference. The pythoncapi-compat project can be used to get `PyWeakref_GetRef()` on Python 3.12 and older. (Contributed by Victor Stinner in gh-105927.)

Pending Removal in Python 3.14

- The `ma_version_tag` field in `PyDictObject` for extension modules (**PEP 699**; gh-101193).
- Creating `immutable types` with mutable bases (gh-95388).
- Functions to configure Python's initialization, deprecated in Python 3.11:
  - `PySys_SetArgvEx()`: Set `PyConfig.argv` instead.
  - `PySys_SetArgv()`: Set `PyConfig.argv` instead.
  - `Py_SetProgramName()`: Set `PyConfig.program_name` instead.
  - `Py_SetPythonHome()`: Set `PyConfig.home` instead.

  The `Py_InitializeFromConfig()` API should be used with `PyConfig` instead.
- Global configuration variables:
  - `Py_DebugFlag`: Use `PyConfig.parser_debug` instead.
  - `Py_VerboseFlag`: Use `PyConfig.verbose` instead.
  - `Py_QuietFlag`: Use `PyConfig.quiet` instead.
  - `Py_InteractiveFlag`: Use `PyConfig.interactive` instead.
  - `Py_InspectFlag`: Use `PyConfig.inspect` instead.
  - `Py_OptimizeFlag`: Use `PyConfig.optimization_level` instead.
  - `Py_NoSiteFlag`: Use `PyConfig.site_import` instead.
  - `Py_BytesWarningFlag`: Use `PyConfig.bytes_warning` instead.
  - `Py_FrozenFlag`: Use `PyConfig.pathconfig_warnings` instead.
  - `Py_IgnoreEnvironmentFlag`: Use `PyConfig.use_environment` instead.
  - `Py_DontWriteBytecodeFlag`: Use `PyConfig.write_bytecode` instead.
  - `Py_NoUserSiteDirectory`: Use `PyConfig.user_site_directory` instead.
  - `Py_UnbufferedStdioFlag`: Use `PyConfig.buffered_stdio` instead.
  - `Py_HashRandomizationFlag`: Use `PyConfig.use_hash_seed` and `PyConfig.hash_seed` instead.
  - `Py_IsolatedFlag`: Use `PyConfig.isolated` instead.
  - `Py_LegacyWindowsFSEncodingFlag`: Use `PyPreConfig.legacy_windows_fs_encoding` instead.
  - `Py_LegacyWindowsStdioFlag`: Use `PyConfig.legacy_windows_stdio` instead.
  - `Py_FileSystemDefaultEncoding`: Use `PyConfig.filesystem_encoding` instead.
  - `Py_HasFileSystemDefaultEncoding`: Use `PyConfig.filesystem_encoding` instead.
  - `Py_FileSystemDefaultEncodeErrors`: Use `PyConfig.filesystem_errors` instead.
  - `Py_UTF8Mode`: Use `PyPreConfig.utf8_mode` instead. (see `Py_PreInitialize()`)

  The `Py_InitializeFromConfig()` API should be used with `PyConfig` instead.

Pending Removal in Python 3.15

- The `PyImport_ImportModuleNoBlock()`: Use `PyImport_ImportModule()` instead.
- `PyWeakref_GetObject()` and `PyWeakref_GET_OBJECT()`: Use `PyWeakref_GetRef()` instead.
- `Py_UNICODE` type and the `Py_UNICODE_WIDE` macro: Use `wchar_t` instead.
- Python initialization functions:

  - `PySys_ResetWarnOptions()`: Clear `sys.warnoptions` and `warnings.filters` instead.
  - `Py_GetExecPrefix()`: Get `sys.base_exec_prefix` and `sys.exec_prefix` instead.
  - `Py_GetPath()`: Get `sys.path` instead.
  - `Py_GetPrefix()`: Get `sys.base_prefix` and `sys.prefix` instead.
  - `Py_GetProgramFullPath()`: Get `sys.executable` instead.
  - `Py_GetProgramName()`: Get `sys.executable` instead.
  - `Py_GetPythonHome()`: Get `PyConfig.home` or the `PYTHONHOME` environment variable instead.

Pending removal in Python 3.16

- The bundled copy of `libmpdec`.

Pending Removal in Future Versions

The following APIs are deprecated and will be removed, although there is currently no date scheduled for their removal.

- `Py_TPFLAGS_HAVE_FINALIZE`: Unneeded since Python 3.8.
- `PyErr_Fetch()`: Use `PyErr_GetRaisedException()` instead.
- `PyErr_NormalizeException()`: Use `PyErr_GetRaisedException()` instead.
- `PyErr_Restore()`: Use `PyErr_SetRaisedException()` instead.
- `PyModule_GetFilename()`: Use `PyModule_GetFilenameObject()` instead.
- `PyOS_AfterFork()`: Use `PyOS_AfterFork_Child()` instead.
- `PySlice_GetIndicesEx()`: Use `PySlice_Unpack()` and `PySlice_AdjustIndices()` instead.
- `PyUnicode_AsDecodedObject()`: Use `PyCodec_Decode()` instead.
- `PyUnicode_AsDecodedUnicode()`: Use `PyCodec_Decode()` instead.
- `PyUnicode_AsEncodedObject()`: Use `PyCodec_Encode()` instead.
- `PyUnicode_AsEncodedUnicode()`: Use `PyCodec_Encode()` instead.
- `PyUnicode_READY()`: Unneeded since Python 3.12
- `PyErr_Display()`: Use `PyErr_DisplayException()` instead.
- `_PyErr_ChainExceptions()`: Use `_PyErr_ChainExceptions1()` instead.
- `PyBytesObject.ob_shash` member: call `PyObject_Hash()` instead.
- `PyDictObject.ma_version_tag` member.
- Thread Local Storage (TLS) API:
  - `PyThread_create_key()`: Use `PyThread_tss_alloc()` instead.
  - `PyThread_delete_key()`: Use `PyThread_tss_free()` instead.
  - `PyThread_set_key_value()`: Use `PyThread_tss_set()` instead.
  - `PyThread_get_key_value()`: Use `PyThread_tss_get()` instead.
  - `PyThread_delete_key_value()`: Use `PyThread_tss_delete()` instead.
  - `PyThread_ReInitTLS()`: Unneeded since Python 3.7.

11 Build Changes

- `arm64-apple-ios` and `arm64-apple-ios-simulator` are both now **PEP 11** tier 3 platforms. (*PEP 730* written and implementation contributed by Russell Keith-Magee in gh-114099.)
- `aarch64-linux-android` and `x86_64-linux-android` are both now **PEP 11** tier 3 platforms. (*PEP 738* written and implementation contributed by Malcolm Smith in gh-116622.)
- `wasm32-wasi` is now a **PEP 11** tier 2 platform. (Contributed by Brett Cannon in gh-115192.)
- `wasm32-emscripten` is no longer a **PEP 11** supported platform. (Contributed by Brett Cannon in gh-115192.)
- Building CPython now requires a compiler with support for the C11 atomic library, GCC built-in atomic functions, or MSVC interlocked intrinsics.
- Autoconf 2.71 and aclocal 1.16.5 are now required to regenerate the `configure` script. (Contributed by Christian Heimes in gh-89886 and by Victor Stinner in gh-112090.)
- SQLite 3.15.2 or newer is required to build the `sqlite3` extension module. (Contributed by Erlend Aasland in gh-105875.)
- CPython now bundles the mimalloc library by default. It is licensed under the MIT license; see mimalloc license. The bundled mimalloc has custom changes, see gh-113141 for details. (Contributed by Dino Viehland in gh-109914.)
- The `configure` option `--with-system-libmpdec` now defaults to `yes`. The bundled copy of `libmpdec` will be removed in Python 3.16.
- Python built with `configure --with-trace-refs` (tracing references) is now ABI compatible with the Python release build and debug build. (Contributed by Victor Stinner in gh-108634.)
- On POSIX systems, the pkg-config (`.pc`) filenames now include the ABI flags. For example, the free-threaded build generates `python-3.13t.pc` and the debug build generates `python-3.13d.pc`.
- The `errno`, `fcntl`, `grp`, `md5`, `pwd`, `resource`, `termios`, `winsound`, `_ctypes_test`, `_multiprocessing.posixshmem`, `_scproxy`, `_stat`, `_statistics`, `_testconsole`, `_testimportmultiple` and `_uuid` C extensions are now built with the limited C API. (Contributed by Victor Stinner in gh-85283.)

12 Porting to Python 3.13

This section lists previously described changes and other bugfixes that may require changes to your code.

12.1 Changes in the Python API

- *PEP 667* introduces several changes to the semantics of `locals()` and `f_locals`:
  - Calling `locals()` in an optimized scope now produces an independent snapshot on each call, and hence no longer implicitly updates previously returned references. Obtaining the legacy CPython behavior now requires explicit calls to update the initially returned dictionary with the results of subsequent calls to `locals()`. Code execution functions that implicitly target `locals()` (such as `exec` and `eval`) must be passed an explicit namespace to access their results in an optimized scope. (Changed as part of **PEP 667**.)
  - Calling `locals()` from a comprehension at module or class scope (including via `exec` or `eval`) once more behaves as if the comprehension were running as an independent nested function (i.e. the local variables from the containing scope are not included). In Python 3.12, this had changed to include the local variables from the containing scope when implementing **PEP 709**. (Changed as part of **PEP 667**.)
  - Accessing `FrameType.f_locals` in an optimized scope now returns a write-through proxy rather than a snapshot that gets updated at ill-specified times. If a snapshot is desired, it must be created explicitly with `dict` or the proxy's `.copy()` method. (Changed as part of **PEP 667**.)

- `functools.partial` now emits a `FutureWarning` when used as a method. The behavior will change in future Python versions. Wrap it in `staticmethod()` if you want to preserve the old behavior. (Contributed by Serhiy Storchaka in gh-121027.)
- An `OSError` is now raised by `getpass.getuser()` for any failure to retrieve a username, instead of `ImportError` on non-Unix platforms or `KeyError` on Unix platforms where the password database is empty.
- The value of the `mode` attribute of `gzip.GzipFile` is now a string (`'rb'` or `'wb'`) instead of an integer (`1` or `2`). The value of the `mode` attribute of the readable file-like object returned by `zipfile.ZipFile.open()` is now `'rb'` instead of `'r'`. (Contributed by Serhiy Storchaka in gh-115961.)
- `mailbox.Maildir` now ignores files with a leading dot (`.`). (Contributed by Zackery Spytz in gh-65559.)
- `pathlib.Path.glob()` and `rglob()` now return both files and directories if a pattern that ends with "`**`" is given, rather than directories only. Add a trailing slash to keep the previous behavior and only match directories.
- The `threading` module now expects the `_thread` module to have an `_is_main_interpreter()` function. This function takes no arguments and returns `True` if the current interpreter is the main interpreter.

  Any library or application that provides a custom `_thread` module must provide `_is_main_interpreter()`, just like the module's other "private" attributes. (gh-112826.)

12.2 Changes in the C API

- `Python.h` no longer includes the `<ieeefp.h>` standard header. It was included for the `finite()` function which is now provided by the `<math.h>` header. It should now be included explicitly if needed. Remove also the `HAVE_IEEEFP_H` macro. (Contributed by Victor Stinner in gh-108765.)
- `Python.h` no longer includes these standard header files: `<time.h>`, `<sys/select.h>` and `<sys/time.h>`. If needed, they should now be included explicitly. For example, `<time.h>` provides the `clock()` and `gmtime()` functions, `<sys/select.h>` provides the `select()` function, and `<sys/time.h>` provides the `futimes()`, `gettimeofday()` and `setitimer()` functions. (Contributed by Victor Stinner in gh-108765.)
- On Windows, `Python.h` no longer includes the `<stddef.h>` standard header file. If needed, it should now be included explicitly. For example, it provides `offsetof()` function, and `size_t` and `ptrdiff_t` types. Including `<stddef.h>` explicitly was already needed by all other platforms, the `HAVE_STDDEF_H` macro is only defined on Windows. (Contributed by Victor Stinner in gh-108765.)
- If the `Py_LIMITED_API` macro is defined, `Py_BUILD_CORE`, `Py_BUILD_CORE_BUILTIN` and `Py_BUILD_CORE_MODULE` macros are now undefined by `<Python.h>`. (Contributed by Victor Stinner in gh-85283.)
- The old trashcan macros `Py_TRASHCAN_SAFE_BEGIN` and `Py_TRASHCAN_SAFE_END` were removed. They should be replaced by the new macros `Py_TRASHCAN_BEGIN` and `Py_TRASHCAN_END`.

  A `tp_dealloc` function that has the old macros, such as:

```
static void
mytype_dealloc(mytype *p)
{
    PyObject_GC_UnTrack(p);
    Py_TRASHCAN_SAFE_BEGIN(p);
    ...
    Py_TRASHCAN_SAFE_END
}
```

  should migrate to the new macros as follows:

```
static void
mytype_dealloc(mytype *p)
{
    PyObject_GC_UnTrack(p);
    Py_TRASHCAN_BEGIN(p, mytype_dealloc)
```

(continues on next page)

(continued from previous page)

```
    ...
    Py_TRASHCAN_END
}
```

Note that `Py_TRASHCAN_BEGIN` has a second argument which should be the deallocation function it is in. The new macros were added in Python 3.8 and the old macros were deprecated in Python 3.11. (Contributed by Irit Katriel in gh-105111.)

- *PEP 667* introduces several changes to frame-related functions:
  - The effects of mutating the dictionary returned from `PyEval_GetLocals()` in an optimized scope have changed. New dict entries added this way will now *only* be visible to subsequent `PyEval_GetLocals()` calls in that frame, as `PyFrame_GetLocals()`, `locals()`, and `FrameType.f_locals` no longer access the same underlying cached dictionary. Changes made to entries for actual variable names and names added via the write-through proxy interfaces will be overwritten on subsequent calls to `PyEval_GetLocals()` in that frame. The recommended code update depends on how the function was being used, so refer to the deprecation notice on the function for details.
  - Calling `PyFrame_GetLocals()` in an optimized scope now returns a write-through proxy rather than a snapshot that gets updated at ill-specified times. If a snapshot is desired, it must be created explicitly (e.g. with `PyDict_Copy()`), or by calling the new `PyEval_GetFrameLocals()` API.
  - `PyFrame_FastToLocals()` and `PyFrame_FastToLocalsWithError()` no longer have any effect. Calling these functions has been redundant since Python 3.11, when `PyFrame_GetLocals()` was first introduced.
  - `PyFrame_LocalsToFast()` no longer has any effect. Calling this function is redundant now that `PyFrame_GetLocals()` returns a write-through proxy for optimized scopes.
- Python 3.13 removed many private functions. Some of them can be replaced using these alternatives:
  - `_PyDict_Pop()`: `PyDict_Pop()` or `PyDict_PopString()`;
  - `_PyDict_GetItemWithError()`: `PyDict_GetItemRef()`;
  - `_PyErr_WriteUnraisableMsg()`: `PyErr_FormatUnraisable()`;
  - `_PyEval_SetTrace()`: `PyEval_SetTrace()` or `PyEval_SetTraceAllThreads()`;
  - `_PyList_Extend()`: `PyList_Extend()`;
  - `_PyLong_AsInt()`: `PyLong_AsInt()`;
  - `_PyMem_RawStrdup()`: `strdup()`;
  - `_PyMem_Strdup()`: `strdup()`;
  - `_PyObject_ClearManagedDict()`: `PyObject_ClearManagedDict()`;
  - `_PyObject_VisitManagedDict()`: `PyObject_VisitManagedDict()`;
  - `_PyThreadState_UncheckedGet()`: `PyThreadState_GetUnchecked()`;
  - `_PyTime_AsSecondsDouble()`: `PyTime_AsSecondsDouble()`;
  - `_PyTime_GetMonotonicClock()`: `PyTime_Monotonic()` or `PyTime_MonotonicRaw()`;
  - `_PyTime_GetPerfCounter()`: `PyTime_PerfCounter()` or `PyTime_PerfCounterRaw()`;
  - `_PyTime_GetSystemClock()`: `PyTime_Time()` or `PyTime_TimeRaw()`;
  - `_PyTime_MAX`: `PyTime_MAX`;
  - `_PyTime_MIN`: `PyTime_MIN`;
  - `_PyTime_t`: `PyTime_t`;
  - `_Py_HashPointer()`: `Py_HashPointer()`;
  - `_Py_IsFinalizing()`: `Py_IsFinalizing()`.

The pythoncapi-compat project can be used to get most of these new functions on Python 3.12 and older.

13 Regression Test Changes

- Python built with `configure --with-pydebug` now supports a `-X presite=package.module` command-line option. If used, it specifies a module that should be imported early in the lifecycle of the interpreter, before `site.py` is executed. (Contributed by Łukasz Langa in gh-110769.)

14 Notable changes in 3.13.1

14.1 sys

- The previously undocumented special function `sys.getobjects()`, which only exists in specialized builds of Python, may now return objects from other interpreters than the one it's called in.

15 Notable changes in 3.13.4

15.1 os.path

- The *strict* parameter to `os.path.realpath()` accepts a new value, `os.path.ALLOW_MISSING`. If used, errors other than `FileNotFoundError` will be re-raised; the resulting path can be missing but it will be free of symlinks. (Contributed by Petr Viktorin for **CVE 2025-4517**.)

15.2 tarfile

- `data_filter()` now normalizes symbolic link targets in order to avoid path traversal attacks. (Contributed by Petr Viktorin in gh-127987 and **CVE 2025-4138**.)
- `extractall()` now skips fixing up directory attributes when a directory was removed or replaced by another kind of file. (Contributed by Petr Viktorin in gh-127987 and **CVE 2024-12718**.)
- `extract()` and `extractall()` now (re-)apply the extraction filter when substituting a link (hard or symbolic) with a copy of another archive member, and when fixing up directory attributes. The former raises a new exception, `LinkFallbackError`. (Contributed by Petr Viktorin for **CVE 2025-4330** and **CVE 2024-12718**.)
- `extract()` and `extractall()` no longer extract rejected members when `errorlevel()` is zero. (Contributed by Matt Prodani and Petr Viktorin in gh-112887 and **CVE 2025-4435**.)

Index

C

Common Vulnerabilities and Exposures
    CVE 2023-27043, 13
    CVE 2023-52425, 19
    CVE 2024-4030, 15, 17
    CVE 2024-12718, 42
    CVE 2025-4138, 42
    CVE 2025-4330, 42
    CVE 2025-4435, 42
    CVE 2025-4517, 42

E

environment variable
    PYTHON_BASIC_REPL, 5
    PYTHON_COLORS, 4, 5, 13
    PYTHON_CPU_COUNT, 15
    PYTHON_FROZEN_MODULES, 9
    PYTHON_GIL, 6, 7
    PYTHON_HISTORY, 9
    PYTHON_JIT, 7
    PYTHON_PERF_JIT_SUPPORT, 9
    PYTHONHOME, 36, 38
    PYTHONLEGACYWINDOWSFSENCODING, 24, 28

P

Python Enhancement Proposals
    PEP 11, 8, 39
    PEP 11#tier-2, 5
    PEP 11#tier-3, 4
    PEP 587, 35
    PEP 590, 36
    PEP 594, 3, 19
    PEP 602, 5
    PEP 626, 27
    PEP 667, 4, 8, 36, 39
    PEP 669, 4, 31
    PEP 688#current-options, 28
    PEP 696, 4
    PEP 699, 37
    PEP 702, 4, 19
    PEP 703, 3, 4, 7
    PEP 705, 4, 18
    PEP 709, 39
    PEP 719, 3
    PEP 730, 4, 8
    PEP 737, 34
    PEP 738, 4, 8
    PEP 742, 4, 18
    PEP 744, 3, 4, 8
PYTHON_BASIC_REPL, 5
PYTHON_COLORS, 4, 5, 13
PYTHON_CPU_COUNT, 15
PYTHON_FROZEN_MODULES, 9
PYTHON_GIL, 6, 7
PYTHON_HISTORY, 9
PYTHON_JIT, 7
PYTHON_PERF_JIT_SUPPORT, 9
PYTHONHOME, 36, 38
PYTHONLEGACYWINDOWSFSENCODING, 24, 28

R

RFC
    RFC 5280, 16